
GLPI Agent

Release 1.17

GLPI Project, Teclib'

Apr 02, 2026

CONTENTS

1	Compatibility and main purpose	3
1.1	Installation	3
1.1.1	Windows	3
1.1.1.1	Windows installer	3
1.1.1.2	Command line parameters	4
1.1.1.3	Winget	11
1.1.1.4	Large Installations	12
1.1.1.5	Contributions	13
1.1.1.5.1	Glpi-Agent Monitor	13
1.1.2	MacOS	13
1.1.3	GNU/Linux	14
1.1.3.1	Linux AppImage installer	14
1.1.3.1.1	Linux AppImage Installer	14
1.1.3.2	Snap	20
1.1.3.3	Linux Perl Installer	20
1.1.3.4	Unofficial repositories	21
1.1.4	From sources	21
1.1.4.1	Tests	21
1.1.4.2	Perl dependencies	22
1.1.4.3	Offline installations	22
1.1.4.4	Other dependencies	22
1.2	Configuration	22
1.2.1	System location	22
1.2.2	Parameter syntax	23
1.2.3	Available parameters	23
1.2.4	Task-specific parameters	28
1.3	Usage	30
1.3.1	Running the agent	30
1.3.1.1	HTTP interface	30
1.3.1.2	Command line	30
1.3.1.2.1	Windows	30
1.3.1.2.2	OS X	31
1.3.1.2.3	Others	31
1.3.2	Execution modes	31
1.3.2.1	Managed mode	32
1.3.2.2	Half-managed mode	32
1.3.2.3	Autonomous mode	32
1.3.3	Offline usage	33
1.3.3.1	Agent execution	33
1.3.3.2	Result import	33

	1.3.3.2.1	GLPI Interface	33
	1.3.3.2.2	glpi-injector	33
	1.3.3.2.3	curl	33
1.3.4		Usage caution	33
	1.3.4.1	Concurrent executions	33
	1.3.4.2	Multiple execution targets	34
1.4	Tasks		34
	1.4.1	Remote inventory	34
	1.4.1.1	Overview	34
	1.4.1.1.1	Pre-requisite	34
	1.4.1.2	Setup	35
	1.4.1.2.1	Targets	35
	1.4.1.2.2	Remote registration	35
	1.4.1.2.3	Managing remotes	35
	1.4.1.3	Running	36
	1.4.1.3.1	Automatic execution	36
	1.4.1.3.2	Command-line execution	36
	1.4.1.3.3	Without remote registration	36
	1.4.1.3.4	Performance with X remotes defined	36
	1.4.1.3.5	Modes	37
	1.4.1.4	Caveats	37
	1.4.2	Network discovery	37
	1.4.2.1	Overview	38
	1.4.2.2	Running	38
	1.4.2.2.1	Pre-requisite	38
	1.4.2.2.2	Command-line execution	39
	1.4.2.2.3	Efficiency concerns	39
	1.4.2.3	Advanced Configuration	39
	1.4.2.4	Troubleshooting	39
	1.4.3	Network inventory	40
	1.4.3.1	Overview	40
	1.4.3.2	Running	40
	1.4.3.2.1	Pre-requisite	40
	1.4.3.2.2	Command-line execution	41
	1.4.3.3	Advanced Configuration	41
	1.4.3.4	Troubleshooting	41
1.5	HTTP Interface		41
	1.5.1	Security	42
	1.5.2	Plugins	42
	1.5.2.1	Basic Authentication Server Plugin	42
	1.5.2.1.1	Setup	42
	1.5.2.1.2	Configuration	42
	1.5.2.1.3	Basic Authentication use cases	44
	1.5.2.2	SSL Server Plugin	44
	1.5.2.2.1	Setup	44
	1.5.2.2.2	Configuration	45
	1.5.2.3	Proxy Server Plugin	46
	1.5.2.3.1	Setup	46
	1.5.2.3.2	Configuration	46
	1.5.2.3.3	Use cases	48
	1.5.2.4	Inventory Server Plugin	50
	1.5.2.4.1	Setup	50
	1.5.2.4.2	Configuration	50
	1.5.2.4.3	Inventory request	52

	1.5.2.4.4	Use cases	52
	1.5.2.5	Test Server Plugin	53
	1.5.2.5.1	Setup	53
	1.5.2.5.2	Configuration	53
	1.5.2.6	ToolBox interface	54
	1.5.2.6.1	Setup	55
	1.5.2.6.2	Credentials	56
	1.5.2.6.3	IP Ranges	58
	1.5.2.6.4	Scheduling	61
	1.5.2.6.5	Inventory tasks	64
	1.5.2.6.6	Configuration files	67
1.6		IDS Databases	68
	1.6.1	SNMP device IDS database	68
1.7		Bug reporting	69
	1.7.1	Problem description	69
	1.7.2	External data	69
	1.7.2.1	WMI queries	70
	1.7.2.2	Registry extract	70
	1.7.2.3	dmidecode output	70
	1.7.2.4	snmpwalk output	70
1.8		Man pages	71
	1.8.1	glpi-agent	71
	1.8.1.1	NAME	71
	1.8.1.2	SYNOPSIS	71
	1.8.1.3	DESCRIPTION	74
	1.8.1.4	OPTIONS	74
	1.8.1.4.1	Target definition options	74
	1.8.1.4.2	Target scheduling options	74
	1.8.1.4.3	Task selection options	75
	1.8.1.4.4	Inventory task specific options	75
	1.8.1.4.5	ESX task specific options	77
	1.8.1.4.6	Package deployment task specific options	77
	1.8.1.4.7	Server target specific options	77
	1.8.1.4.8	Web interface options	78
	1.8.1.4.9	Server authentication	78
	1.8.1.4.10	Logging options	78
	1.8.1.4.11	Configuration options	79
	1.8.1.4.12	Execution mode options	79
	1.8.2	glpi-inventory	80
	1.8.2.1	NAME	80
	1.8.2.2	SYNOPSIS	80
	1.8.2.3	DESCRIPTION	81
	1.8.3	glpi-netdiscovery	81
	1.8.3.1	NAME	81
	1.8.3.2	SYNOPSIS	81
	1.8.3.3	DESCRIPTION	82
	1.8.3.4	OPTIONS	82
	1.8.3.5	EXAMPLES	83
	1.8.4	glpi-netinventory	83
	1.8.4.1	NAME	84
	1.8.4.2	SYNOPSIS	84
	1.8.4.3	DESCRIPTION	84
	1.8.4.4	OPTIONS	84
	1.8.4.5	EXAMPLES	86

1.8.5	glpi-esx	86
1.8.5.1	NAME	86
1.8.5.2	SYNOPSIS	86
1.8.5.3	EXAMPLES	87
1.8.5.4	DESCRIPTION	87
1.8.5.5	LIMITATION	87
1.8.5.6	SECURITY	87
1.8.6	glpi-injector	87
1.8.6.1	NAME	87
1.8.6.2	SYNOPSIS	87
1.8.6.3	DESCRIPTION	88
1.8.7	glpi-remote	88
1.8.7.1	NAME	89
1.8.7.2	SYNOPSIS	89
1.8.7.3	DESCRIPTION	90
1.8.7.4	OPTIONS	91
1.8.7.4.1	Target definition options	91
1.8.7.4.2	General options	91
1.8.7.4.3	<i>agent</i> sub-command options	92
1.8.7.4.4	Sub-commands	92
1.8.7.4.5	Environment variables	93
1.8.8	glpi-win32-service	93
1.8.8.1	NAME	93
1.8.8.2	SYNOPSIS	93

2	Documentation license	95
----------	------------------------------	-----------

Welcome to the GLPI Agent Documentations, your comprehensive guide to understanding, installing, configuring, and effectively managing the GLPI Agent. As an integral component within the GLPI Inventory ecosystem, the GLPI Agent plays an important role in simplifying inventory tracking, management and unify asset management processes.

The GLPI Agent is a tool developed and designed to address complex challenges associated with IT asset management. It makes the collection and transmission of critical data from connected devices to the central GLPI Server, ensuring that administrators and IT professionals can maintain a real-time hardware and software photograph of the company or clients. With this information, organizations can make decisions based in real data, reduce downtime, optimize resource allocation, enhance security protocols, plan new purchases and be in compliance with licensing, and security requirements.

This documentation is directed to a diverse audience. It can include system administrators, IT professionals, network engineers, and anyone responsible for tasks of asset and inventory management within the organization. Whether you are an experienced IT specialist looking to harness the full potential of GLPI Agent or a beginner seeking to understand its functionalities, this guide is designed to meet your needs.

This documentation provides a comprehensive coverage of the GLPI Agent, including the full scope of tasks involved in its deployment and management. This embraces step-by-step instructions on installation, detailed insights and hints into configuration settings, practical guidance on usage, and a robust troubleshooting section to help you find and fix common issues. We believe that empowering administrator with the knowledge and tools required to maximize the benefits of the GLPI Agent is the best way to ensure a seamless and efficient asset management experience.

COMPATIBILITY AND MAIN PURPOSE

GLPI Agent is the successor of [FusionInventory Agent](#), since it's based on the same code and it can be easily used in place of any FusionInventory agent.

- It is used to run automatic inventory and works with [GLPI ITSM software tool](#).
- It also supports running few other tasks like package deployment, information collect, network devices discovery and inventory, ESX remote inventory.
- It also supports agentless inventory through its `remoteinventory` task.
- It was developed to allow you to automatically run tasks that can currently only be run manually or using `glpi-inventory` plugin.
- On server-side, it only depends on GLPI starting from GLPI 10 version.

Attention

For older GLPI version (9.5 or below), it also depends on [FusionInventory for GLPI plugin](#).

1.1 Installation

The latest release is available from [our github releases page](#).

Note

In the case you're replacing [FusionInventory agent](#) with GLPI Agent, you should first uninstall **FusionInventory agent** before installing **GLPI Agent**. You can use the same dedicated configuration if you placed it in `conf.d` configuration subfolder.

Hint

Nightly builds are also available from [our dedicated GLPI-Agent Nightly Builds page](#).

1.1.1 Windows

1.1.1.1 Windows installer

By default, the installer will bring you to the graphical user interface unless you use the `/i /quiet` options, calling it from command line.

C: > GLPI-Agent-1.17-x64.msi /quiet SERVER=<URL>

or:

C: > msexec /i GLPI-Agent-1.17-x64.msi /quiet SERVER=<URL>

All options can be defined in several places; the last taking precedence on all others:

- default values,
- values from the current GLPI Agent installation,
- values from the command line,
- values from the graphical installer.

Note

When using command line parameters, you should keep in mind:

- parameters beginning with a slash are indeed `msiexec.exe` options,
- an equal sign is always required for other parameters: `TAG=prod`,
- options names are case-sensitive,
- options values are *not* case-sensitive, unless specified,
- the equal sign must not be preceded or followed with a space character: `LOCAL = C:\temp` is incorrect,
- if a value contains a space, it must be surrounded with single ' or double quotes "",
- if you want to set a empty value, put an empty string (`LOCAL=` or `LOCAL=""`).

Attention

Don't use **PowerShell** as commandline interpreter: it changes some environment context the installer doesn't handle well.

1.1.1.2 Command line parameters

/i

Specify this is a normal installation. This is indeed a `msiexec.exe` option.

When used with `msiexec.exe`, it must be used just before the MSI package path.

/quiet

Silent installation. This is indeed a `msiexec.exe` option. (By default: No)

ADD_FIREWALL_EXCEPTION=1

Adds GLPI Agent to the Windows Firewall exception list. (By default: 0 for No)

ADDITIONAL_CONTENT=filename (needs MSI installer >= 1.3)

Specifies an XML file whose content will be automatically merged with output. If inventory format is JSON, you can also specify a JSON file from which content base node will be merged. (By default: empty)

ADDLOCAL=feature[,feature[...]]

This parameter permits to select features to install. (By default: empty string)

The *feature* can take the following values:

- **ALL**: All included tasks are selected for installation

- `feat_AGENT`: to restrict to the minimum installation with agent and Inventory task
- `feat_NETINV`: to select NetDiscovery and NetInventory tasks for installation
- `feat_DEPLOY`: to select Deploy task for installation
- `feat_COLLECT`: to select Collect task for installation
- `feat_ESX`: to select ESX task for installation
- `feat_WOL`: to select WakeOnLan task for installation (indeed not supported)

The base feature is `feat_AGENT` which is always selected and includes Inventory task. Since v1.8, no other task is selected by default.

AGENTMONITOR=1 (needs MSI installer >= v1.5)

Install Glpi-AgentMonitor (See [GLPI-Agent Monitor project](#)).

Only applicable if GLPI-Agent is installed as a service (i.e. `EXECMODE=1`).

AGENTMONITOR_NEWTICKET_SCREENSHOT=0|1 (needs MSI installer >= v1.11)

Sets Glpi-AgentMonitor flag to enable or not automatic screenshot on new ticket request. (By default: 1)

Only applicable if GLPI-Agent is installed as a service and Glpi-AgentMonitor installed (i.e. `EXECMODE=1` and `AGENTMONITOR=1`).

AGENTMONITOR_NEWTICKET_URL=URL (needs MSI installer >= v1.8)

Sets Glpi-AgentMonitor URL for new ticket feature. (By default: empty string)

You only need to set this URL if the Glpi-AgentMonitor guessed one is not the expected one.

Only applicable if GLPI-Agent is installed as a service and Glpi-AgentMonitor installed (i.e. `EXECMODE=1` and `AGENTMONITOR=1`).

BACKEND_COLLECT_TIMEOUT=180

Timeout for task Inventory modules execution. (By default: 180 seconds)

It is also used as based timeout for RemoteInventory timeouts computation. You may experience datas loss during RemoteInventory task if this value is too short.

CA_CERT_DIR=pathname

Absolute path to the standard certificate directory of certificate authorities (CA). (By default: empty)

The use of this parameter is incompatible with the use of the `CA_CERT_FILE` parameter. The `CA_CERT_DIR` and `CA_CERT_FILE` parameters are mutually exclusive.

A *standard certificate directory* must contain the certificate files corresponding to different certificate authorities in Privacy Enhanced Mail (PEM) format, and their names must correspond to the hash value of the certificate's *subject* field, with extension `.0`.

For example, if you want to include the certificate file `FICert_Class1.crt` in the directory `pathname`, you must calculate the hash value of the certificate's *subject* field using, for example, OpenSSL

```
C: openssl.exe x509 -in C:_Class1.crt -subject_hash -noout
b760f1ce
```

and afterwards, move or copy the certificate file to the directory `pathname` with the name `b760f1ce.0`.

```
C: > move.exe C:_Class1.crt pathname\b760f1ce.0
```

CA_CERT_FILE=filename

Full path to the certificates file of certification authorities (CA). (By default: empty)

The use of this parameter is incompatible with the use of the `CA_CERT_DIR` parameter. The `CA_CERT_DIR` and `CA_CERT_FILE` parameters are mutually exclusive.

filename must have extension `.pem` (Privacy Enhanced Mail) and can contain one or more certificates of certificate authorities. To concatenate multiple certificate files into one file you can make use, for example, of the command *copy*.

```
C:                > copy.exe FICert_Class1.crt+FICert_Class2.crt FICerts.pem
```

DEBUG=level

Sets the debug level of the agent. (By default: `0`)

level can take one of the following values:

- `0`: Debug off
- `1`: Normal debug
- `2`: Full debug

DELAYTIME=limit

Sets an initial delay before first contact with a remote destination (see `SERVER`). This delay is calculated at random between *limit*/2 and *limit* seconds. (Default: `3600` seconds)

This parameter is ignored for remote destinations after the first contact with each one, in favor of the specific server parameter (`PROLOG_FREQ` or Contact expiration).

The `DELAYTIME` parameter comes into play only if GLPI Agent runs in *server mode* (see `EXECMODE`).

ESX_ITEMTYPE=type (needs MSI installer >= 1.13)

Sets ESX inventory itemtype to support GLPI 11+ genericity feature. (By default: empty)

When empty, asset itemtype in ESX inventory will still be set to `Computer` to keep compatibility with GLPI 10.

See also *esx-itemtype* configuration for more details.

EXECMODE=value

Sets the agent execution mode. (By default: `1`)

mode can take one of the following values:

- `1` for Service: The agent runs as a Windows Service (always running)
- `2` for Task: The agent runs as a Windows Task (runs at intervals)
- `3` for Manual: The agent doesn't run automatically (no Service, no Task)

The mode `Service` is known also as *server mode*.

The mode `Task` is only available on Windows XP (or higher) and Windows Server 2003 (or higher) operative systems.

FULL_INVENTORY_POSTPONE=value (needs MSI installer >= v1.8)

Sets the expected number of full inventory postpone. (By default: `14`)

See *full-inventory-postpone* configuration for more details.

GLPI_VERSION=version

Set expected GLPI version to enable specifically required features. (By default: empty)

This can be required if you miss a GLPI-version dependent feature in few context like generating locally an inventory or not using native inventory.

See *glpi-version* configuration for more details.

HTML=1

Save the inventory as HTML instead of XML or JSON. (By default: `0` for No)

The `HTML` parameter comes into play only if you have also indicated a value for the `LOCAL` parameter.

HTTPD_IP=ip

IP address by which the embedded web server should listen. (By default: 0.0.0.0)

HTTPD_PORT=port

IP port by which the embedded web server should listen. (By default: 62354)

HTTPD_TRUST={ip|range|hostname}[,{ip|range|hostname}...]

Trusted IP addresses that do not require authentication token by the integrated web server. (By default: 127.0.0.1/32)

ip is an IP address in dot-decimal notation (ex. "127.0.0.1") or in CIDR notation (ex. "127.0.0.1/32")

range is an IP address range in dot-decimal notation (ex. "192.168.0.0 - 192.168.0.255" or "192.168.0.0 + 255") or in CIDR notation (ex. "192.168.0.0/24")

hostname is the name of a host (ex. "itms.acme.org")

Keep in mind that HTTPD_TRUST does not have to include the hostname part of those URIs that are set up in SERVER because they are tacitly included. The following is an example, both configurations are equal:

```
... HTTPD_TRUST="127.0.0.1/32,itms.acme.org" \
  SERVER="http://itms.acme.org/glpi/"
```

```
... HTTPD_TRUST="127.0.0.1/32" \
  SERVER="http://itms.acme.org/glpi/"
```

INSTALLDIR=pathname

Sets the installation base directory of the agent. (By default: C:\Program Files\GLPI-Agent)

pathname must be an absolute path.

ITEMTYPE=type (needs MSI installer >= 1.13)

Sets inventory itemtype to support GLPI 11+ genericity feature. (By default: empty)

When empty, asset itemtype in inventory will still be set to Computer to keep compatibility with GLPI 10.

See also *itemtype* configuration for more details.

JSON=0 (needs MSI installer >= 1.3)

Don't save the local inventory as JSON instead of XML. (By default: 1 for Yes)

The JSON parameter comes into play only if you have also indicated a value for the LOCAL parameter.

LAZY=1

Contact server only if the server expiration delay has been reached. (By default: 1)

This option is only used if you set EXECMODE=2 to use Windows Task scheduling.

LISTEN=1 (needs MSI installer >= 1.3)

Force agent to always listen for requests on httpd interface, even when no target is defined with server or local option. (By default: 0 for No)

Very useful in combination with [Inventory Server plugin](#).

LOCAL=pathname

Writes the results of tasks execution into the given directory. (By default: empty)

You must indicate an absolute pathname or an empty string (""). If you indicate an empty string, the results of tasks execution will not be written locally.

You can use the LOCAL and SERVER options simultaneously.

LOGFILE=filename

Writes log messages into the file *filename*. (By default: C:\Program Files\GLPI-Agent\logs\glpi-agent.log)

You must indicate a full path in *filename*. The LOGFILE parameter comes into play only if you have also indicated file as a value of the LOGGER parameter, which is the default.

LOGFILE_MAXSIZE=size

Sets the maximum size of logfile (see LOGFILE) to *size* in MBytes. (By default: 4 MBytes)

LOGGER=backend[, backend]

Sets the logger backends. (By default: file)

backend can take any of the following values:

- file: Sends the log messages to a file (see LOGFILE)
- stderr: Sends the log messages to the console

NO_CATEGORY=category[, category[...]]

Do not inventory the indicated categories of elements. (By default: empty)

category can take any value listed by the following command:

```
C: > "C: Files-Agent-agent" --list-categories
```

NO_COMPRESSION=1 (needs MSI installer >= 1.3)

Disable compression when exchanging informations with GLPI Server. (By default: 0)

NO_HTTPD=1

Disables the embedded web server. (By default: 0)

NO_P2P=1

Do not use peer to peer to download files. (By default: 0)

NO_SSL_CHECK=1

Do not check server certificate. (By default: 0)

NO_TASK=task[, task[...]]

Disables the given tasks. (By default: empty)

task can take any of the following values:

- Deploy: Task Deploy
- ESX: Task ESX
- Inventory: Task Inventory
- NetDiscovery: Task NetDiscovery
- NetInventory: Task NetInventory
- WakeOnLan: Task WakeOnLan

If you indicate an empty string (""), all tasks will be executed.

OAUTH_CLIENT_ID=id (needs MSI installer >= v1.10)

Specifies the GLPI OAuth2 client ID for server authentication.

 Attention

OAuth2 authentication support in GLPI for inventory submission is planned to be release with next GLPI major release, GLPI 11. The feature can be tested with [GLPI main version nightly builds](#).

OAuth clients credentials have to be created in the dedicated **Configuration** panel in **GLPI 11** and greater with **"Client credentials"** as **Grants** value and **"inventory"** as **Scope** value.

OAuth_CLIENT_SECRET=secret (needs MSI installer >= v1.10)

Specifies the GLPI OAuth2 client secret for server authentication.

PASSWORD=password

Uses *password* as password for server authentication. (By default: empty)

The PASSWORD comes into play only if you have also indicated a value for the SERVER parameter.

PROXY=URI

Uses *URI* as HTTP/S proxy server. (By default: empty)

QUICKINSTALL=1

Don't ask for detailed configurations during graphical install. (By default: 0)

REINSTALL=feat_AGENT

Use this option only in the case you need to change the agent configuration using the same installer. (Not used by default)

REQUIRED_CATEGORY=category[, category[...]] (needs MSI installer >= v1.13)

Sets categories to always include in partial inventory when postponing a full inventory. (By default: empty)

See *required-category* configuration for more details.

REMOTE=remote:definition (needs MSI installer >= 1.3)

Specify a remote inventory definition to be used by *Remote inventory* task. (By default: empty)

REMOTE_WORKERS=max (needs MSI installer >= v1.5)

Set the maximum number of remote inventory to process at the same time. (By default: 1)

RUNNOW=1

Launches the agent immediately after its installation. (By default: 0)

SCAN_HOMEDIRS=1

Allows the agent to scan home directories for virtual machines. (By default: 0)

SCAN_PROFILES=1

Allows the agent to scan softwares installed in users profiles. (By default: 0)

SERVER=URI[, URI[...]]

Sends results of tasks execution to given servers. (By default: empty)

If you indicate an empty string (""), the results of tasks execution will not be written remotely.

You can use the SERVER and LOCAL parameters simultaneously.

SNMP_RETRIES=max (needs MSI installer >= v1.9)

Set the maximum number of time a SNMP request could be retried again on no device response. (By default: 0)

Only used by NetDiscovery and NetInventory tasks.

SSL_CERT_FILE=filename (needs MSI installer >= 1.3)

Specifies the file containing SSL client certificate to use when connecting to server target or for WinRM remote inventory. (By default: empty)

SSL_KEY_FILE=filename (needs MSI installer >= 1.16)

Specifies the file containing SSL client private key to use when connecting to server target or for WinRM remote inventory. (By default: empty)

SSL_FINGERPRINT=fingerprint (needs MSI installer >= 1.3)

Specifies the fingerprint of the ssl server certificate to trust.

The fingerprint to use can be retrieved in agent log by temporarily enabling *no-ssl-check* option.

SSL_KEYSTORE=[none|key[,key[,...]]] (needs MSI installer >= 1.11)

Can be used to disable keystore if set to *none*.

Specifies a comma-separated list of Keystore stores. See *ssl-keystore* for possible key values and explanation.

TAG=tag

Marks the computer with the tag *tag* . (By default: empty)

TASKS=task[,task[,...]]

Plan tasks in the given order. (By default: empty)

Not listed tasks won't be planned during a run, unless , ... is specified at the end.

task can take any of the following values:

- Deploy: Task Deploy
- ESX: Task ESX
- Inventory: Task Inventory
- NetDiscovery: Task NetDiscovery
- NetInventory: Task NetInventory
- WakeOnLan: Task WakeOnLan

If you indicate an empty string (""), all tasks will be executed. If you indicate , ... at the end, all not listed tasks will be added in any order. You can indicate a task more than one time if this makes sens.

TASK_DAILY_MODIFIER=modifier

Daily task schedule modifier. (By default: 1 day)

modifier can take values between 1 and 365, both included.

The TASK_DAILY_MODIFIER parameter comes into play only if you have also indicated *daily* as value of the TASK_FREQUENCY option.

TASK_FREQUENCY=frequency

Frequency for task schedule. (By default: hourly)

frequency can take any of the following values:

- minute: At minute intervals (see TASK_MINUTE_MODIFIER parameter)
- hourly: At hour intervals (see TASK_HOURLY_MODIFIER parameter)
- daily: At day intervals (see TASK_DAILY_MODIFIER parameter)

TASK_HOURLY_MODIFIER=modifier

Hourly task schedule modifier. (By default: 1 hour)

modifier can take values between 1 and 23, both included.

The TASK_HOURLY_MODIFIER parameter comes into play only if you have also indicated *hourly* as value of the TASK_FREQUENCY parameter.

TASK_MINUTE_MODIFIER=modifier

Minute task schedule modifier. (By default: 15 minutes)

modifier can take the any value from 1 to 1439.

The TASK_MINUTE_MODIFIER parameter comes into play only if you have also indicated *minute* as value of the TASK_FREQUENCY parameter.

TIMEOUT=180

Sets the limit time (in seconds) to connect with the server. (By default: 180 seconds)

The TIMEOUT parameter comes into play only if you have also indicated a value for the SERVER parameter.

USER=user

Uses *user* as user for server authentication. (By default: empty)

The USER parameter comes into play only if you have also indicated a value for the SERVER parameter.

VARDIR=pathname

Sets the vardir base directory of the agent. (By default: C:\Program Files\GLPI-Agent\var)

This parameter can be used when the agent is installed in a shared storage.

pathname must be an absolute path.

The installer integrates its native, although reduced but recent, version of [Strawberry Perl](#) including recent [OpenSSL](#) support.

You can download the latest [GLPI Agent installer](#) or [current nightly build](#). It is available for 64 bits systems and provides a graphical interface as well as command line facilities.

Note

Since GLPI-Agent 1.8, no more installer is provided for 32 bits systems. The latest version supporting 32 systems is 1.7.3.

By default, it will perform a graphical installation, unless you use the `msiexec /i` and `/quiet` options. All installer parameters are described in [Windows installer](#) dedicated page.

Note

All graphical installer options are related to a command line one. Check [Windows installer](#) if you need help.

1.1.1.3 Winget

You can also use [Winget tool](#) to install or update GLPI-Agent on Windows 10 and Windows 11.

Note

Winget provides 1.7.3 x64 & x86 packages. Starting from 1.8, only x64 package is provided.

You can check glpi-agent availability from the command line:

```
C: > winget search glpi-agent
```

During first installation, you can trigger interactive installation running:

```
C: > winget install -i glpi-agent
```

To install without configuration or update using current configuration, just run:

```
C: > winget install glpi-agent
```

Note

As it is installed at machine level and requires administrative privileges, an **UAC prompt** should be triggered if you don't start the installation from an administrative console.

During first installation, you can use `--custom` option to configure `glpi-agent` as expected, for example for a quiet installation. You can use all parameters described in [Windows installer](#) page:

```
C: > winget install glpi-agent --custom="SERVER='https://my-glpi-server/' RUNNOW=1"
```

Note

If you didn't configure during first installation, you still can edit configuration in registry under `HKEY_LOCAL_MACHINE\SOFTWARE\GLPI-Agent` registry key. Don't forget to restart the service if installed as a service (by default, it is).

1.1.1.4 Large Installations

Consider a scenario where the GLPI Agent application needs to be installed on large and diverse range of Windows systems. A VBScript can be useful in this scenario to perform the following tasks:

- Install it silently taking variables from the installation script previously configured.
- Check for the presence of FusionInventory and OCS Inventory agents and uninstall them if you need it.
- Configure application settings based on [command line parameters that can be explored here](#).
- Log installation progress and errors for review and analysis.

A VBScript (Visual Basic Script) is provided to deploy the installer on a network: `glpi-agent-deployment.vbs`.

In this script you'll find some variables that can be changed to your environment needs (uncomment what is commented and you need. Comment what you don't need.):

- **GLPI Agent Version** hereby named `SetupVersion`
- Setup the **Location** from where the script will download the MSI hereby named `SetupLocation`. It can be a HTTP or HTTPS url, a CIFS or local folder. The default is to use the github release page url.
- Setup the **Architecture of your systems** (if you need it to be x86, x64, or if you want to let it be installed according to the system Architecture) hereby named `SetupArchitecture`
- **Setup Options** from the [command line parameters](#):
 - It is recommended to keep the `/quiet` parameter so the user will not be bothered with wizard or command line messages or windows
 - If you want to follow all the steps of installations, don't add the `/quiet` parameter.
- **Reconfigure**:
 - You will just set this up to Yes if the current installed agent has the same version you have configured on the `SetupVersion` above. This option, when activated, is going to reconfigure the new options for the same Agent.
 - It's useful when you just need to change a parameter like GLPI SERVER url, for example.
- **Repair**:
 - This option will unregister and register the MSI service DLL, and reinstall it with the options selected on the script.

- It works just when the Setup is still installed.
- **Verbose:**
 - It Enables or Disables
- **Run uninstall:**
 - Here you can uncomment the deprecated agents you want the script to uninstall (FusionInventory Agent or OCSInventory Agent)

Hint

Adding this VBS Script to a Computer GPO to run on startup of computers, usually works better, since there are some users with no installation rights. The msi download should be accessible for every computer that needs to execute it. If you are using a local network sharing, or a regular Github URL, you must make sure the computers are able to download it from the original source.

1.1.1.5 Contributions

1.1.1.5.1 Glpi-Agent Monitor

Note

This application requires that you have enabled the embedded http server

Leonardo Bernardes published **Glpi-Agent Monitor** tool which provides a systray icon to survey the agent status and permit to ask a tasks run.

You can download it from the [Glpi-Agent Monitor](#) project release page. You simply have to download the exe into any folder (the Glpi-Agent installation folder is just fine) and start it.

If you want to add it to all users auto-started softwares, you can directly install it into the **C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp** folder.

Hint

Glpi-Agent Monitor has been integrated into **GLPI-Agent MSI installer** and can be enabled using *AGENTMONITOR=1* on the MSI commandline. When installed this way, you don't need to copy it manually and it starts when a user is logs in.

1.1.2 MacOS

The installer integrates its native, although reduced but recent, version of [Perl](#) including recent [OpenSSL](#) support.

Get the latest `.pkg` package from [our releases page](#) or the [nightly build page](#). After installing it, you'll have to configure the agent to your needs by creating a dedicated `.cfg` file under the `/Applications/GLPI-Agent/etc/conf.d` folder.

You can for example create a `local.cfg` file and:

- add the `server = GLPI_URL` line to point to your GLPI server,
- eventually set `debug = 1` to generate some debug in logs,
- set a tag like `tag = MyLovelyTag`.

 Hint

A MacOSX installation video tutorial is available here: [GLPI Agent Demonstration - macOS Monterey - Apple M1](#)

1.1.3 GNU/Linux

We support major distros as we provides generic packages for **RPM** and **DEB** based distros as well if they supports **Snap** packaging. You can install required packages after getting them from [our github releases page](#) or the [nightly build page](#).

 Hint

When possible, prefer to use our [linux perl installer](#) as it supports **RPM** and **DEB** based distros. There's a version also including the **Snap** package. The linux installer accepts few options to configure the agent so it can simplify manual or automatic installation. It also can be handy for tools like [Puppet](#) or [Ansible](#).

1.1.3.1 Linux AppImage installer

1.1.3.1.1 Linux AppImage Installer

Linux [AppImage](#) installer is another way to install glpi-agent on linux. It has the advantage to work on any linux supporting AppImage format (most linux distros since years).

To install the agent, just run:

```
$ chmod +x glpi-agent-1.17-x86_64.AppImage
$ sudo ./glpi-agent-1.17-x86_64.AppImage --install --server <URL>
```

At this point, if you obtain an error explaining AppImages require FUSE to run., you still can try a [manual install](#).

 Hint

On Ubuntu, starting with Ubuntu 22.04, fuse3 is installed, but AppImage requires libfuse2. So you should install it with:

```
$ sudo apt install libfuse2
```

 Caution

As [reported by MarcSamD](#), if you have a message telling `libcrypt.so.1` is missing, you will need to install a compatibility package providing that library. On RPM based distro, you probably just need to install `libxcrypt-compat` package.

To upgrade the agent if you still installed glpi-agent linux appimage, just run:

```
$ chmod +x glpi-agent-1.17-x86_64.AppImage
$ sudo ./glpi-agent-1.17-x86_64.AppImage --upgrade
```

By default, the agent is installed under `/usr/local/bin` and dedicated configuration is created under `/etc/glpi-agent/conf.d`.

To uninstall the agent, you simply can run:

```
$ sudo glpi-agent-uninstall
```

Or to also clean any data in `/etc/glpi-agent` and `/var/lib/glpi-agent`:

```
$ sudo glpi-agent-uninstall --clean
```

Installer parameters

Hint

You can retrieve all available parameters running:

```
$ ./glpi-agent-1.17-x86_64.AppImage --help
```

--install

Install the agent by coping AppImage to install path, creating binding scripts for GLPI Agent commands. It copies default configurations and creates dedicated configuration to setup the agent with configuration parameters.

-i --installpath=PATH

Define PATH as installation folder. (By default: `/usr/local/bin`)

--upgrade

Try to just upgrade the currently installed agent, keeping configuration and trying to restart the agent.

--reinstall

Like upgrade but involves `--clean` option to make a clean install, resetting the configuration.

--uninstall

Try to uninstall currently installed Glpi Agent AppImage.

--config=PATH

When installing, copy the given configuration file in `/etc/glpi-agent/conf.d`

--clean

Clean everything when uninstalling or before installing.

--runnow

Run agent tasks after installation.

--service

Install GLPI Agent AppImage as a service. This option is selected by default.

--no-service

Don't install GLPI Agent as a service.

--cron=SCHED

Install agent as cron task (no by default). SCHED can only be set to daily or hourly.

--wait=DELAY (available since GLPI-Agent v1.10)

delay the tasks run by DELAY seconds when installed with `--cron` option.

--version

Output the installer version and exit.

-S --silent

Make installer silent.

-h --help

Output help and exit.

--script=SCRIPT

Run embedded script in place of installer.

--perl

Run embedded perl.

Configuration parameters

All configuration options are documented in [Configuration](#) page and in the *glpi-agent* man page.

Target options

Most importantly, at least one target definition option is mandatory when installing GLPI Agent.

-s --server=URI

send tasks result to a server

-l --local=PATH

write tasks results locally in a file

Scheduling options

--delaytime=LIMIT

maximum delay before running target tasks the first time

--lazy

do not contact the target before next scheduled time

Task selection options

--no-task=TASK[,TASK]...

do not run given task

--tasks=TASK1[,TASK]...[,...]

run given tasks in given order

Inventory task specific options

--no-category=CATEGORY

do not include given categories in inventory

--scan-homedirs

scan user home directories

--scan-profiles

scan user profiles

--html

save inventory as HTML

--json

save inventory as JSON

--force

always send data to server

--backend-collect-timeout=TIME

timeout for inventory modules execution

Remote inventory task specific options

--remote=REMOTE
setup remote for which request remote inventory

Deploy task specific options

--no-p2p
do not use peer to peer to download files

Network options

--proxy=PROXY
proxy address

--user=USER
user name for server authentication

--password=PASSWORD
password for server authentication

--ca-cert-dir=DIRECTORY
CA certificates directory

--ca-cert-file=FILE
CA certificate file

--ssl-cert-file=FILE
Client certificate file

--ssl-key-file=FILE
Client private key file

--ssl-fingerprint=FINGERPRINT
Fingerprint of SSL certificate to trust

--no-ssl-check
do not check server SSL certificate

-C --no-compression
do not compress communication with server

--timeout=TIME
connection timeout

Web interface options

--no-httpd
disable embedded web server

--httpd-ip=IP
local network ip to listen on

--httpd-port=PORT
network port to listen on

--httpd-trust=IP
trust given IPs and IP ranges

--listen
enable listener target if required

Logging options

- logger=BACKEND***
logger backend
- logfile=FILE***
log file
- logfile-maxsize=SIZE***
maximum size of the log file
- logfacility=FACILITY***
syslog facility
- color***
use color in the console
- debug***
enable debug mode

General options

- conf-reload-interval=TIME***
number of seconds between two configuration reloading
- t --tag=TAG***
add given tag to inventory results
- vardir=PATH***
use specified path as storage folder for agent persistent datas

Manual install

In the case, FUSE is not installed on the system and you can't or don't want to install it, you still can install GLPI Agent manually by following these steps:

- Extract AppImage content:

```
$ ./glpi-agent-1.17-x86_64.AppImage --appimage-extract
```

This will extract the content into a `squashfs-root` subfolder.
- Copy the `squashfs-root` folder to a dedicated place:

```
$ [ -d /opt ] || sudo mkdir /opt  
$ sudo rm -rf /opt/glpi-agent  
$ sudo cp -r squashfs-root /opt/glpi-agent
```
- Run the AppRun from copied folder as installer:

```
$ sudo /opt/glpi-agent/AppRun --install --server <URL>
```

To uninstall after a manual install, you need to run:

```
$ sudo /usr/local/bin/glpi-agent-uninstall  
$ sudo rm -rf /opt/glpi-agent
```


Portable installation

It is possible to use AppImage installer to create a portable linux glpi agent environment.

Creation

Here are the step to install such environment:

1. Download [glpi-agent-portable.sh](#)
2. Download a GLPI Agent AppImage
3. Copy script and AppImage to a dedicated folder, for example at the root of an USB key or a network shared folder
4. Make script and AppImage executable with:


```
$ sudo chmod +x glpi-agent-portable.sh glpi-agent*.AppImage
```
5. Run one time `glpi-agent-portable.sh` to setup the environment. This will create a `etc/` and a `var/` sub-folder and all scripts at the same level. Don't remove `glpi-agent-portable.sh` and AppImage.


```
$ sudo ./glpi-agent-portable.sh
```
6. Create a `.cfg` file under `etc/conf.g` to configure your agent or create dedicated script which start expected glpi-agent scripts with expected parameters.

You're now ready to use the linux portable agent.

Note

As installed scripts are using `--vardir=PATH` option, agent deviceid will be defined depending on the current computer hostname. So you can safely run it on different computers. The deviceid will even be reused later if you run it again on a given computer.

Hint

You can also specify AppImage to use by defining `APPIMAGE` environment variable.

Upgrade

It is really simple to upgrade a portable installation:

1. Remove old AppImage from the folder
2. Download and copy the newer AppImage
3. Make AppImage executable


```
$ sudo chmod +x glpi-agent*.AppImage
```

Hint

When not sure or linux perl installer doesn't support your distro, try Linux AppImage installer.

See [Linux AppImage installer dedicated page](#)

1.1.3.2 Snap

The **Snapcraft Snap** package integrates its native, although reduced but recent, version of **Perl** including recent **OpenSSL** support.

If your system support **Snap**, you can simply install the agent with the **snap** command after getting the **Snap** package from [our releases page](#) or the [nightly build page](#). Then, you just have to run:

```
$ snap install --classic --dangerous GLPI-Agent-1.17_amd64.snap
```

After installation, you can easily configure the agent with the **set snap** sub-command:

```
$ snap set glpi-agent server=http://my-glpi-server/
```

Any supported glpi-agent option can be set this way. If you need to unset a configuration parameter, just set it empty:

```
$ snap set glpi-agent tag=
```

Note

You won't find the package in the [Snapcraft](#) store as their standard policies are too restrictive for GLPI Agent features and requirements.

1.1.3.3 Linux Perl Installer

Attention

The **linux installer** main requirement is the **perl** command.

It also requires one of the following command, depending on the targeted system:

- **dnf** for recent **RPM** based systems
- **yum** for previous generation of **RPM** based systems
- **apt** for **DEB** based systems like Debian & Ubuntu
- **snap** for other systems supporting [Snapcraft Snap](#) packages

We also provide a dedicated linux installer which includes all the packages we build (**RPM** & **DEB**) and eventually the [snap](#) one. On supported distros (**DEB** & **RPM** based), the installer will also eventually try to enable third party repositories, like EPEL on CentOS if they are required.

The installer is a simple perl script. It supports various options to configure the agent during installation. If no options are specified, the default is for the installer to configure glpi-agent to not scan user home directories or profiles, to run inventory modules with a 30 minute timeout, to allow HTTP requests to port 62354 only from the GLPI server, to setup the glpi-agent as a service and to install only the Computer Inventory and Remote Inventory tasks. You can check all supported options by running:

```
$ perl glpi-agent-1.17-linux-installer.pl --help
```

or if you use the installer embedding **snap** package:

```
$ perl glpi-agent-1.17-with-snap-linux-installer.pl --help
```

If your GNU/Linux distro is not supported, you still can [install it from sources](#).

1.1.3.4 Unofficial repositories

Attention

Unofficial repositories are not supported by GLPI-Agent editor. Use them at your own risk.

Thanks to ligenix, a COPR repository can be used to install glpi-agent on Fedora 35 & 36, CentOS Stream 8 & 9, EPEL 7, 8 & 9: [ligenix/enterprise-glpi10](https://copr.fedoraproject.org/repo/pkgs/ligenix/enterprise-glpi10/)

1.1.4 From sources

Note

We strongly recommend the use of *GNU tar* because some file path length are greater than 100 characters. Some tar version will silently ignore those files.

First, you need to extract the source and change the current directory.

```
$ tar xzf GLPI-Agent-1.17.tar.gz
$ cd GLPI-Agent-1.17
```

Executing `Makefile.PL` will verify all the required dependencies are available and prepare the build tree.

```
$ perl Makefile.PL
```

If you don't want to use the default directory (`/usr/local`), you can use the `PREFIX` parameter:

```
$ perl Makefile.PL PREFIX=/opt/glpi-agent
```

Note

At this point, you may have some missing required modules. See *PERL Dependencies* section for installing them. Once this is done, run the same command again.

You now can finish the installation. Here again we recommend *GNU make* (*gmake*):

```
$ make
$ make install
```

1.1.4.1 Tests

Note

The tests suite requires some additional dependencies like `Test::More`.

GLPI agent comes with a test-suite. You can run it with this command:

```
$ make test
```

1.1.4.2 Perl dependencies

The easiest way to install [perl](#) dependencies is to use [cpanminus](#) script, running:

```
$ cpanm .
```

You can use the `--notest` flag if you are brave and want to skip the tests suite execution for each install perl module.

1.1.4.3 Offline installations

Note

This requires the `cpanminus` script to be installed.

First grab the tarball from the website and extract it:

```
$ tar xzf GLPI-Agent-1.17.tar.gz
$ cd GLPI-Agent-1.17
```

We use `cpanm` to fetch and extract the dependencies in the `extlib` directory:

```
$ cpanm --pureperl --installdeps -L extlib --notest .
```

If this command fails with an error related to `Params::Validate`, then just run this last command:

```
$ cpanm --installdeps -L extlib --notest .
```

Now you can copy the directory to another machine and run the agent this way:

```
$ perl -Iextlib/lib/perl5 -Ilib glpi-agent
```

1.1.4.4 Other dependencies

On Solaris/SPARC, you must install [sneep](#) and record the Serial Number with it.

On Windows, we use an additional `dmidecode` binary shipped in the windows MSI package to retrieve many information not available otherwise, including fine-grained multi-cores CPUs identification. Unfortunately, this binary is not reliable enough to be used on Windows 2003, leading to less precise inventories.

On Linux, `lspci` will be used to collect PCI, AGP, PCI-X, ... information.

1.2 Configuration

1.2.1 System location

On Unix, the agent reads its configuration from a configuration file named `agent.cfg`, whose location depends of the installation method:

- `/etc/glpi-agent/agent.cfg` on FHS (File System Hierarchy) compliant systems
- `/Applications/GLPI-Agent/etc/agent.cfg` on MacOS X pkg

More globally, you'll find that file in the GLPI Agent installation directory.

It is strongly discouraged to change this file, as you will probably loose your configuration on update (especially if you use a linux or MacOS X package).

Just ensure the `include conf.d/` is not commented (does not starts with a `#`). Your specific configuration should then go to any `conf.d/*.cfg` file.

On Windows, the agent read its configuration from a registry key, whose location may depends on architecture:

- HKEY_LOCAL_MACHINE\SOFTWARE\GLPI-Agent is the default,
- HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\GLPI-Agent for 32bits agent installed on a 64bits system.

But windows portable version will use `agent.cfg` located under `etc` folder unless `--config=registry` option is used.

1.2.2 Parameter syntax

Most of the configuration options are single-valued; you can use a comma (,) as separator for multi-valued ones:

In configuration file:

```
logger = stderr,file
```

On command-line:

```
$ glpi-agent --logger=stderr,file
$ glpi-agent --logger stderr,file
```

1.2.3 Available parameters

Note

Most configuration options can be specified on command line ; this will override configuration file values in that case.

The only required configuration parameter is an execution target, which depends on the mode you will use:

- **server**: a server URL, such as `https://my-glpi-server/`, or `https://my-glpi-server/front/inventory.php`,
- **local**: full path for local directory, like `/tmp/inventory`.

Hint

If you're using **GLPI 10+**, you may want to use **GlpInventory plugin** as a replacement for FusionInventory plugin **netdiscovery**, **netinventory**, **deploy**, **collect** and **esx** tasks management until this will be integrated in core.

Caution

About the server URL to use as **server** parameter, it will depends on you server and plugins installation:

- If you're using **GLPI 9.5.x** with **FusionInventory for GLPI plugin 9.5+3.0**:
Your server URL should look like: `https://my-glpi-server/plugins/fusioninventory/`
- If you're using **GLPI 10+**, there are few cases regarding **GlpInventory plugin** usage:
 1. If you're not using **GlpInventory plugin**:
Your server URL should look like: `https://my-glpi-server/`, or `https://my-glpi-server/front/inventory.php`
Using `https://my-glpi-server/` may cause issues fixed in GLPI 10.0.6. If you use an older version, it may be better to use the full URL for now.

2. If you have installed **GlpInventory plugin** via **Marketplace**:

Your server URL should look like: `https://my-glpi-server/marketplace/glpinventory/`

3. If you have installed **GlpInventory plugin** manually under `/plugins` GLPI folder:

Your server URL should look like: `https://my-glpi-server/plugins/glpinventory/`

Caution

In the case you installed your GLPI on IIS, you may need to add `index.php` at the end of your server parameter to use the right handler. But as explained by [@SteadEXE in an issue](#), you can fix this problem updating your IIS configuration.

server

Specifies the server to use both as a controller for the agent, and as a recipient for task execution output.

If the given value start with `http://` or `https://`, it is assumed to be an URL, and used directly. Otherwise, it is assumed to be an hostname, and interpreted as `http://hostname/inventory`.

Multiple values can be specified, using a comma as a separator.

Warning

Using multiple targets implies multiple executions of the same inventory ; this is not just a matter of targets. This can lead to different results, see [Multiple execution targets](#).

local

Write the results of tasks execution locally. Exact behaviour according to given path:

- if parameter is a directory, a file will be created therein
- if parameter is a file, it will be used directly
- if parameter is `-`, **STDOUT** will be used

Multiple values can be specified, using a comma as a separator.

Warning

Using multiple targets implies multiple executions of the same inventory ; this is not just a matter of targets. This can lead to different results, see [Multiple execution targets](#).

include

This directive can only be used from a configuration file and permits to specify a file or a path from where to load any `*.cfg` files.

The default is `conf.d` to load any `<INSTALLDIR>/etc/conf.d/*.cfg` file.

conf-reload-interval

Automatically reload agent configuration after the given delay in seconds. The default is 0 which value just disables the feature.

delaytime

Specifies the upper limit, in seconds, for the initial delay before contacting the control server. The default is 3600.

The actual delay is computed randomly between $TIME / 2$ and $TIME$ seconds.

This directive is used for initial contact only, and ignored thereafter in favor of server-provided value in response from prolog or Contact request.

lazy

Do not contact the control server before next scheduled time.

This directive is used when the agent is run in the foreground (not as a daemon) only.

no-task

Disables given task.

Multiple values can be specified, using a comma as a separator.

tasks

Define tasks to run and in which order.

Using `...` string in a list means run all remaining tasks.

Multiple values can be specified, using a comma as a separator.

proxy

Specifies the URL of the HTTP proxy to use. By default, the agent uses `HTTP_PROXY` environment variable.

Since GLPI Agent v1.13, you can set `proxy` value to `none` to even not use `HTTP_PROXY` environment variable.

user

Specifies the user to use for HTTP authentication on the server.

password

Specifies the password to use for HTTP authentication on the server.

oauth-client-id (Available since GLPI Agent v1.10)

Specifies the GLPI OAuth2 client ID for server authentication.

! Attention

OAuth2 authentication support in GLPI for inventory submission is planned to be release with next GLPI major release, GLPI 11. The feature can be tested with [GLPI main version nightly builds](#).

OAuth clients credentials have to be created in the dedicated **Configuration** panel in **GLPI 11** and greater with **"Client credentials"** as **Grants** value and **"inventory"** as **Scope** value.

oauth-client-secret (Available since GLPI Agent v1.10)

Specifies the GLPI OAuth2 client secret for server authentication.

ca-cert-dir

Specifies the directory containing indexed Certification Authority (CA) certificates.

This directory must contain the certificate files corresponding to different certificate authorities in Privacy Enhanced Mail (PEM) format. The file name of each certificate file must match the hash value of the certificate's *subject* field and use the `.0` extension.

You can obtain the hash value of the certificate's *subject* field and copy the `CA.crt` certificate to the expected place following this snippet:

```
$ CA_CERT_DIR=/etc/glpi-agent/ca-cert-dir
$ openssl x509 -in CA.crt -subject_hash -noout
b760f1ce
* cp -a CA.crt $CA_CERT_DIR/b760f1ce.0
```

ca-cert-file

Specifies the file containing aggregated Certification Authority (CA) certificates.

ssl-cert-file

Specifies the file containing SSL client certificate to use when connecting to server target or for WinRM remote inventory.

ssl-key-file

Specifies the file containing SSL client private key to use when connecting to server target or for WinRM remote inventory.

ssl-fingerprint (Available since GLPI Agent v1.3)

Specifies the fingerprint of the ssl server certificate to trust.

The fingerprint to use can be retrieved in agent log by temporarily enabling *no-ssl-check* option.

ssl-keystore (Available since GLPI Agent v1.11)

This option is only usable on Windows or MacOSX.

Keystore support on Windows and Keychain support on MacOSX are enabled by default to provide a way to authenticate SSL GLPI server if CA certificate or server certificate is integrated there.

It takes as argument a string which can be a list separated by commas:

- **none**: just disable keystore support on Windows or keychain support on MacOSX
- Only on Windows, any combination of the following **case-sensitive** keys:
 - My, CA, Root for default machine store
 - User-My, User-CA, User-Root for machine user store
 - Service-My, Service-CA, Service-Root for machine service store
 - Enterprise-My, Enterprise-CA, Enterprise-Root for machine enterprise store
 - GroupPolicy-My, GroupPolicy-CA, GroupPolicy-Root for machine group policy store
- Only on MacOSX, the user ('root' as a daemon) keychain will be used.
 - You can force to use the system SSL CA keychain by using the **system-ssl-ca** key (Available since GLPI Agent v1.12)

This can help if your certificate authority is not known by glpi-agent default SSL store based on [Mozilla::CA](#) but it is by system SSL CA. GLPI-Agent will use [certutil command](#) and [security find-certificates command](#) to extract certificates from related store.

This option will be ignored if used at the same time than one of **ca-cert-dir**, **ca-cert-file** or **ssl-fingerprint** options.

no-ssl-check

Disables server SSL certificate validation. The default is 0 (false).

timeout

Specifies a timeout, in seconds, for server connections.

no-httpd

Disables the embedded web interface, used to receive execution requests from the GLPI server or serve httpd plugins. The default is 0 (false).

httpd-ip

Specifies the network interface to use for the embedded web interface. The default is to use all available ones.

httpd-port

Specifies the network port to use for the embedded web interface. The default is 62354.

httpd-trust

Specifies which IP address should be trusted, for execution requests. The default is to only accept requests from the control servers.

All formats supported by [Net::IP](#) can be used (IP addresses, IP addresses ranges, networks in CIDR notation), as well as hostnames.

Multiple values can be specified, using a comma as a separator.

logger

Specifies the logger backend to use. The possible values are:

- file: log messages in a file.
- stderr: log messages directly in the console.
- syslog: log messages through the local syslog server.

Multiple values can be specified, using a comma as a separator.

logfile

Specifies the file to use for the file logger backend.

logfile-maxsize

Specifies the maximum size for the log file, in MB. When the max size is reached, the file is truncated. The default is unlimited.

logfacility

Specifies the syslog facility to use for the syslog logger backend. The default is LOG_USER.

color

Enables color display for the stderr logger backend.

This directive is used on Unix only.

debug

Specifies the level of verbosity for log content. The possible values are:

- 0: basic agent processing
- 1: extended agent processing
- 2: messages exchanged with the server and activates traces from Net::SSLeay if used

no-compression

Disable compression when exchanging informations with GLPI Server. The default is to compress data.

This directive is only supported when server option is set.

listen

Force agent to always listen for requests on httpd interface, even when no target is defined with server or local option.

This directive does nothing if server or local option is set.

vardir

Set dedicated `vardir` path as agent storage. The default is `<INSTALLDIR>/var` on MacOSX, win32 or source install and generally `/var/lib/glpi-agent` on linux/unix when installed with a package.

1.2.4 Task-specific parameters

tag

Specifies an arbitrary string to add to output. This can be used as an additional decision criteria on server side.

This directive is only for inventory or esx task only.

no-category

Disables given category in output. The possible values can be listed running `glpi-agent --list-categories`. Some available categories:

- printer
- software
- environment
- process
- user

Multiple values can be specified, using a comma as a separator.

This directive is used for inventory task only.

full-inventory-postpone (Available since GLPI Agent v1.8)

Specifies the number of times the agent can decide to report a full inventory and produce a partial inventory which only includes changed categories. The default is 14.

This feature is only supported when using json as inventory format so GLPI 10 is required as server. This can reduce a lot the GLPI workload and can help to reduce GLPI server carbon footprint.

It can be set to 0 to disable the feature and to always produce full inventory as before 1.8.

required-category (Available since GLPI Agent v1.13)

This option modifies how full-inventory-postpone generates partial inventory by always including listed categories even if the related section in inventory didn't change since the last inventory. For example, this may be required in the case you're using business rules in GLPI which is based on a category content. Typically, you'll always require the `network` category if you have rules based on ip range analysis.

The possible values can be listed running `glpi-agent --list-categories`.

Multiple values can be specified, using a comma as a separator.

This directive is used for inventory task only and is only relevant if *full-inventory-postpone* is not disabled.

glpi-version (Available since GLPI Agent v1.12)

Specifies for which GLPI server version we should handle tasks

This feature is intended to permit to use features depending on the GLPI server version when GLPI Agent is not aware of this version. If you're using native inventory in json format, you don't need to set this option as the server will tell to GLPI Agent which version is installed on server-side.

This can be the case when you use scripts to locally generate inventory before sending them to GLPI or if you don't use native inventory to send XML files.

itemtype (Available since GLPI Agent v1.13)

Specifies the inventory itemtype to set in inventory and supported by GLPI 11+ genericity feature.

By default, it is empty and itemtype will still be set to `Computer` to keep compatibility with GLPI 10.

The itemtype must be supported in GLPI 11+ or inventory import will be refused. When itemtype in GLPI is, as example, `Server`, itemtype value in `glpi-agent` must be set to `Glpi\CustomAsset\ServerAsset`.

Itemtype can only be set when generating a JSON inventory.

This directive is used for inventory and remoteinventory tasks only.

Note

The suffix `Asset` must always be added at the end of the class name to avoid conflicts with PHP reserved keywords.

additional-content

Specifies an XML file whose content will be automatically merged with output. If inventory format is JSON, you can also specify a JSON file from which `content` base node will be merged.

This directive is used for inventory task only.

scan-homedirs

Enables scanning user home directories for virtual machines (Any OS) or licenses (MacOS X only) . The default is 0 (false).

This directive is used for inventory task only.

scan-profiles

Enables scanning profiles for softwares installation (Win32). The default is 0 (false).

This directive is used for inventory task only.

force

Execute the task, even if not required by the server.

This directive is used for inventory task only.

backend-collect-timeout

Specifies the timeout in seconds for task modules execution. The default is 300.

This directive is used for inventory task only.

esx-itemtype (Available since GLPI Agent v1.13)

Specifies the inventory itemtype to set in ESX inventory and supported by GLPI 11+ genericity feature.

By default, it is empty and itemtype will still be set to `Computer` to keep compatibility with GLPI 10.

The itemtype must be supported in GLPI 11+ or inventory import will be refused. When itemtype in GLPI is, as example, `Esx`, `esx-itemtype` value in `glpi-agent` must be set to `\Glpi\CustomAsset\Esx`.

Itemtype can only be set when generating a JSON inventory.

This directive is used for esx task only.

no-p2p

Disables peer to peer for downloading files.

This directive is used for deploy task only.

html

Output inventory in HTML format.

This directive is used for inventory task and for local target only.

json

Use JSON as inventory format.

This directive is used for inventory task.

remote

Specify a remote inventory definition to be used by *Remote inventory* task.

remote-workers (Available since GLPI Agent v1.5)

Specify the maximum number of remote inventory the agent can process at the same time.

By default, only one remote inventory can be processed at a given time.

assetname-support (Available since GLPI Agent v1.5)

On unix/linux, this option permits to decide how the computer should be named while referencing the computer name. This option can be set to a numeric value:

- 1 (the default) means to use the short name
- 2 means to leave the found hostname unchanged, this can be a Fully Qualified Domain Name (FQDN) or a short name depending on the system
- 3 means to always try to use the FQDN (this support case is available since GLPI Agent v1.7)

MacOSX & Win32 platforms don't support this option.

For remoteinventory, this option is not supported until v1.6.1. And later, the 3 value support for FQDN needs *perl* to be installed on the remote computer and *perl* mode enabled on the defined remote for the RemoteInventory task.

Until v1.6.1, this option only affects the computer name as seen in GLPI Assets list. After, it will also be used to decide how agent reports its *deviceid* to GLPI.

snmp-retries (Available since GLPI Agent v1.9)

Set the maximum number of time a SNMP request could be retried again after no device response. The default is 0.

This directive is used for netdiscovery et netinventory tasks.

Caution

snmp-retries can make snmp inventories more longer to achieve. Use only if you find a device sometime is really not answering in time.

1.3 Usage

1.3.1 Running the agent

1.3.1.1 HTTP interface

If the agent is running as a service or a daemon, its web interface should be accessible at `http://hostname:62354`.

If the machine connecting to this interface is trusted (see [httpd-trust configuration directive](#)), a link will be available to force immediate execution.

1.3.1.2 Command line

Agent can also be executed from command line, using one of available executables.

1.3.1.2.1 Windows

⚠ Attention

Don't use **PowerShell** as commandline interpreter: it changes some environment context the agent doesn't handle well.

Open a command interpreter windows (`cmd.exe`), with administrator privileges (*right click, Run as Administrator*).

Go to `C:\Program files\GLPI-Agent` (adapt path depending on your configuration) folder to run it:

```
C:                                     > cd "C: files-Agent"
C:          Program files             GLPI-Agent> glpi-agent
```

1.3.1.2.2 OS X

Ensure you have rights to run the executable with `sudo` command:

```
$ sudo /Applications/GLPI-Agent.app/bin/glpi-agent
```

1.3.1.2.3 Others

In most of the cases, you should just run (as an administrator):

```
$ glpi-agent
```

1.3.2 Execution modes

"How to run the agent" is not limited to a choice between running it either as a cron task (a scheduled task under Windows) or as a daemon (a service under Windows), because this only makes a difference in the control of the execution schedule, ie when the agent runs: in both cases, the server controls the execution plan, ie what the agent does. In order to control this execution plan on agent side, it is also possible to use a different set of executables. The different possibilities, designed as *execution modes*, are the following:

- **managed mode:** the agent runs continuously in the background, wakes up periodically to execute the tasks required by the server, and may eventually execute out of schedule on server request.
- **half-managed mode:** the agent only runs when launched by a local mechanism (usually an automated scheduling system, such as cron or task scheduler), contacts the servers to ask for a task list, executes them immediately, and stops.
- **autonomous mode:** the agent only runs when launched by a local mechanism, as in previous mode, executes a fixed list of tasks, sends the results to the server, and stops.

This table summarizes who controls what in each mode:

Execution mode	Execution schedule	Execution plan
Managed	Server-side control	Server-side control
Half-managed	Agent-side control	Server-side control
Autonomous	Agent-side control	Agent-side control

The correct mode to use for your needs mainly depends on the role assigned to your GLPI server:

- Is it a trusted management platform, or just a passive information database ? In the first case, you'd rather need server-side control, whereas on the second case, you'd rather need agent-side control

But it also depends on your technical expertise: if everything so far sounds has no meaning for you, no need to go further, just select managed mode. Otherwise, the following presents each mode with additional details.

1.3.2.1 Managed mode

This mode requires the agent to be run as a server process (daemon under Unix, service under Windows). The agent wake-up schedule is controlled from GLPI server, using PROLOG_FREQ or Contact expiration setting. For example, the agent wakes up at a random time between 90% and 100% of this value, ie for 24H, it will executes at sometime between 23 and 24H. Additionally, the server may also initiate additional out-of-schedule executions by sending HTTP requests to the agent.

Example:

```
$ glpi-agent --server http://glpi/ --daemon
```

That's the easiest mode to set up, offering (almost) complete control from a centralized point, fully compatible with all available agent features, and the most flexible in terms of usage.

On the downside, this mode involves having a Perl interpreter loaded in memory permanently, which is insignificant on any modern desktop, but may eventually be a concern in specific memory-constraints scenario, such as IoT or minimal virtual machines. It also involves having a privileged process listening on a network port, unless run with `no-httpd` configuration directive (see [no-httpd configuration](#)).

And the more important: who controls the GLPI servers also controls all assets where an agent is installed, with ability to execute code at anytime, which may involve running arbitrary command with full privileges if related tasks (currently: deploy) are installed AND enabled on agent side. That's the exact purpose of this mode: everything the GLPI server wants, when it wants.

1.3.2.2 Half-managed mode

This mode requires a local triggering mechanism to launch the agent. It may be a scheduling system (cron, task scheduler) to run it automatically and regularly, but it may as well be a user session start script, for instance.

Example:

```
$ glpi-agent --server http://glpi/
```

This mode doesn't consume memory permanently, only during agent execution. However, it is also less flexible, as scheduling can't get changed without reconfiguration. But the server still retains control over execution plan, as the agent asks for a task list when run.

This mode is a compromise between the other modes, with the advantages of the autonomous mode in term of resources usage, and the advantages of the managed mode in term of simplicity and flexibility. Its purpose can get summarized as: everything the GLPI server wants, but only when the agent wants.

Note

As a counter-part, if the system scheduling is planned too often, this may involve an overloading on the GLPI server if a lot of GLPI agents starts to submit requests at the same time. A way to avoid this inconvenient is to enable the [lazy configuration](#) so the GLPI server still decide the time before which the agent doesn't have to run tasks.

```
$ glpi-agent --lazy --server http://glpi/
```

See also [Concurrent executions](#) to use `--wait` option.

1.3.2.3 Autonomous mode

This mode requires a local triggering mechanism to launch the agent, as the half-managed mode. It also has the same benefits for memory usage and reduced security concerns. However, the agent only executes a fixed list of tasks, and the server only receives the execution results, without any control. As sending those results may be done separately, this mode may also be used offline. This is achieved by using specific task-dedicated launchers, instead of the GLPI Agent one.

Deferred upload example:

```
$ glpi-inventory --json > inventory.json
$ glpi-injector --file inventory.json --url http://glpi/
```

Immediate upload example:

```
$ glpi-inventory | curl --data @- http://glpi/
```

This mode is the most complex to set-up, as you have to script the execution of multiple programs, this is not just a matter of configuration. It is also restricted to a limited set of agent tasks, for which a dedicated launcher exists (currently: local inventory, network discovery, network inventory). However, you have a full local control of agent execution.

If you don't trust the GLPI server for any reason (for instance, because it is run by another organization), or if your use case is just to report an inventory regularly, this mode is perfectly suited. It can get summarized as: only what the agent wants, only when the agent wants.

1.3.3 Offline usage

1.3.3.1 Agent execution

Most tasks handled by the agent can be executed directly without server, when it is not available, or for easier troubleshooting.

Most tasks have a dedicated launcher for this purpose. For instance, to execute a local inventory:

```
$ glpi-inventory
```

See man pages for details.

1.3.3.2 Result import

1.3.3.2.1 GLPI Interface

Go to the Administration > Inventory menu, choose the Import tab and upload the inventory file.

1.3.3.2.2 glpi-injector

The agent also has a dedicated executable for result injection:

```
$ glpi-injector --file inventory.json --url http://glpi/
```

See *glpi-injector* man page for details.

1.3.3.2.3 curl

You can also use curl to push an inventory. This can be useful if your Perl installation has no SSL support, for instance:

```
$ curl --header "Content-Type: Application/x-compress" --cacert your-ca.pem -u username:password --data
```

With no SSL check and no authentication:

```
$ curl --header "Content-Type: Application/x-compress" -k --data @/tmp/inventory-file.json https://glpi.
```

1.3.4 Usage caution

1.3.4.1 Concurrent executions

When using managed mode, the server automatically distributes agent executions over time, using random scheduling. However, with other modes, the server doesn't have any control of agent execution schedule, and if they all try to

communicate with it simultaneously, for instance because of a cron task executed on all hosts at the same time, the server may get flooded, and become unable to manage the load.

In order to avoid the issue, either distribute automated task execution over time, or use `--wait` command-line option for `glpi-agent` executable, introducing a random delay before effective execution. For instance:

```
# execute agent daily at random time between 0h00 and 0h30
0 0 * * * /usr/bin/glpi-agent --wait=1800
```

1.3.4.2 Multiple execution targets

Using multiple execution targets (servers or local directories) doesn't mean "execute once, upload the result multiple times", but "tries to execute every available task once for each given target". As a result, there is no guarantee that running an inventory for two different servers, or for one server and for local directory, will produce the exact same result.

The only reliable way currently to produce a single inventory, and transmit the result to multiple targets, is to execute the agent in autonomous mode once, and then upload the results multiple times:

```
$ glpi-inventory --json > inventory.json
$ glpi-injector --file inventory.json --url http://my.first.glpi/
$ glpi-injector --file inventory.json --url http://my.second.glpi/
```

1.4 Tasks

1.4.1 Remote inventory

GLPI Agent supports to run computer inventory remotely, this feature can also be referenced as **Agent-less inventory** for targeted computers.

1.4.1.1 Overview

This task can inventory remote computers via:

- **ssh**: for unix/linux computers
- **WinRM**: for win32 computers

1.4.1.1.1 Pre-requisite

To remotely inventory unix/linux computers supporting ssh, GLPI Agent needs to make network requests on ssh port. The remote ssh user **must** have administration privileges.

For windows computers, WinRM **must** be enabled on targeted computers. You can follow [Microsoft official documentation](#) to enable WinRM. But the short way to enable it with minimal security is to run from an administrator console:

```
C: > winrm quickconfig
C: > winrm set winrm/config/service/auth @Basic="true"
C: > winrm set winrm/config/service @AllowUnencrypted="true"
```

If you're using PowerShell, you need to enclose the configuration value in single quotes, such as this:

```
C: > winrm set winrm/config/service/auth '@Basic="true"'
C: > winrm set winrm/config/service '@AllowUnencrypted="true"'
```

You may probably need to also create a dedicated user with administrative privileges and enable the windows remote management firewall rule.

For WinRM, GLPI Agent must be able to make network http requests on WinRM dedicated ports, by default 5985 for HTTP and 5986 for HTTPS. The remote WinRM user **must** have administration privileges.

Hint

WinRM remote inventory can be run from an agent running on unix/linux platform.

1.4.1.2 Setup

Remote computers will be known as **remotes** on GLPI agent side.

1.4.1.2.1 Targets

A remote will have to be associated to a target which can be a GLPI server or a local directory. The target to use will have to be specified by its alias, like `server0`, `local0` or `server1`. Only previously known targets can be used. So to define a new target, first run `glpi-agent` with the required target.

You can check what are known targets by running:

```
$ glpi-remote list targets
```

1.4.1.2.2 Remote registration

So before running inventory, you'll have to register in GLPI Agent environment remote computers with dedicated credentials. This step can be done by `glpi-remote` script using the **add** sub-command.

This is as simple as running:

```
$ glpi-remote add ssh://admin:pass@192.168.43.237
```

or

```
$ glpi-remote add winrm://admin:pass@192.168.48.250 --target server0
```

Note

When add a remote supporting WinRM, the agent will test provided credential and will fail if something goes wrong. You can avoid this check using `-C` or `--no-check` option.

1.4.1.2.3 Managing remotes

After remotes has been registered, you can list them with the following command:

```
$ glpi-remote list
```

This will provides the locally known remotes:

index	deviceid	url	target	Next
↪run date				
1	WIN-2020-09-23-15-37-52	winrm://glpi-agent:****@192.168.100.138	server0	Tue
↪Nov 9 15:46:51 2021				
2	XPS-2021-11-10-15-10-16	winrm://glpi-agent:****@192.168.100.139	server0	on
↪next agent run				

You can delete a remote giving its listing index:

```
$ glpi-remote delete 1
```

You can update credential by simply *register again* the remote as the script will recognize your are updating an existing **remote**.

⚠ Attention

As of this writing, no solution has still been implemented in GLPI to manage remotes. So everything has to be done from the console.

1.4.1.3 Running

1.4.1.3.1 Automatic execution

When run as a service or a daemon and once remotes are registered against GLPI Agent and associated to a target, the agent will run RemoteInventory task when expected, generate an inventory and submit it to the related server or store it to a local path.

The selected target must be a known target:

- if selected target is `server0`, `server` must be defined in configuration,
- if selected target is `server1`, `server` must be defined with at least 2 URLs as `server1` means to use the second URL,
- if selected target is `local0`, `local` must be set in configuration with an existing path.

1.4.1.3.2 Command-line execution

When GLPI Agent is run from the commandline, it will try to run RemoteInventory task if at least one **remote** is known. It will then select one **remote** and only one to run an inventory but only if its `Next run date` has been set to `on next agent run`.

You can try to run only RemoteInventory task with:

```
$ glpi-agent --logger=stderr --tasks remoteinventory
```

You may have to run again the agent if another **remote** is expected to be inventoried. Just run `glpi-remote list` to verify if a **remote** has to be inventoried.

1.4.1.3.3 Without remote registration

You also can use the `--remote` option of *glpi-agent* to process a remote without registering it:

```
$ glpi-agent --remote=ssh://admin:pass@192.168.43.237 --logger=stderr --tasks remoteinventory
```

`--remote` option can be handy to schedule a remote inventory via crontab or windows job scheduling.

💡 Hint

`--remote` value can be a list of remote url separated by commas. So commas are prohibited in passwords.

1.4.1.3.4 Performance with X remotes defined

By default, only one remote inventory can be run at a time.

Since GLPI-Agent 1.5, you can use `--remote-workers` option to set the maximum number of worker the remote inventory task can run at the same time, each worker processing one remote.

For example, the following command will process 2 remote inventory at the same time:

```
$ glpi-agent --remote-workers=2 --remote=ssh://192.168.43.237,ssh://192.168.77.252 --logger=stderr --ta
```

1.4.1.3.5 Modes

In some context, you may need to change the way remote inventory is processed. In that case, you can configure your remote to use modes.

Modes must be set with the remote url itself to only be applied on one remote. The syntax is similar to the URL query string one by adding `?mode=xxxx` where **xxxx** is the mode to use.

For **winrm**, only one mode can be used to require SSL access to remote: `mode=ssl`.

For example, the following command will process a winrm remote inventory over SSL (default port becomes 5986):

```
$ glpi-agent --remote=winrm://admin:pass@192.168.47.237?mode=ssl --logger=stderr --tasks remoteinventory
```

For **ssh**, 3 modes are available:

1. `mode=perl` can be set if perl is available on the remote to try using it for few specific cases (fqdn and domain),
2. `mode=ssh` can be set to not try to use **libssh2** for remote access,
3. `mode=libssh2` can be set to not try to use **ssh** command access if **libssh2** fails.

You can combine modes. To do so, you just need to concatenate them using the underscore sign as separator: `mode=perl_ssh` or `mode=ssh_perl` are valid syntax

By default, the **ssh** mode is: `mode=libssh2_ssh`. So you don't need to specify both and they are still set if **perl** mode is set. **libssh2** and **ssh** modes only need to be used if you have an issue with the other mode.

For example, the following command will process a ssh remote inventory using only libssh2 and enabling perl mode:

```
$ glpi-agent --remote=ssh://admin:pass@192.168.43.237?mode=perl_ssh --logger=stderr --tasks remoteinven
```

1.4.1.4 Caveats

As the inventory is run remotely, you may not obtain exactly the same inventory as if the agent was run locally.

For **winrm** remotes, the informations may miss or may differ from locally run inventory:

- software installation date,
- bios informations (as we can't run dmidecode),
- devices name, type or description (mostly not localized via winrm),
- databases services.

For **ssh** remotes, the informations may miss or may differ from locally run inventory:

- printers,
- databases services.

1.4.2 Network discovery

A network discovery task aims to scan the the network and reports devices found to the GLPI server, so they can be added to the list of known assets.

Once part of the list of known assets, further information can be retrieved from SNMP enabled devices using [Network inventory](#).

1.4.2.1 Overview

This task uses the following protocols to scan IP networks:

- Arp local table request with `arp -a` command or `ip neighbor show` command
- ICMP Echo (aka ping) with fallback on ICMP Timestamp scan¹
- NetBIOS scan (if `Net::NBName` is available and proper credits provided)
- SNMP scan (if `Net::SNMP` is available, and proper credits provided)

Any device replying to at least one of those protocols will be *discovered*, with minimal information, such as mac address and hostname.

Additionally, if the device replies to SNMP, the agent will attempt to identify it using various methods.

The primary method relies on retrieving the value of the dedicated SNMP variable (`SNMPv2-MIB::sysObjectID.0`), which is a constructor-specific OID identifying a given device model, and comparing it to the agent internal database (the `sysobject.ids` file, described in *SNMP device IDS database*).

If a match is found, model, type and manufacturer are added to the information reported to the GLPI server, allowing simple identification. If no match is found, various heuristics are performed in order to identify the device, with lower reliability.

A secondary method relies on GLPI Agent `MibSupport` feature which permits to implement dedicated perl module for any kind of device. As an example, when the `sysObjectID` is reported as **linux** with the `8072.3.2.10` model-specific suffix, the **LinuxAppliance MibSupport** module usage is triggered and permits to inventory Synology or Ubiquiti devices.

Discovered devices are then reported to the GLPI servers, where import rules are applied. Devices not matching any import criteria will be kept in a server list of ignored devices.

1.4.2.2 Running

1.4.2.2.1 Pre-requisite

The agent performing the task needs to have the **netdiscovery** module installed.

The agent performing the task needs network access the target networks, with forementioned protocols, as well as control access, for SNMP: just being able to send UDP packets to a device is not enough, if this device is configured to ignore them. It is even best to use an agent from the same network than the scanned devices so it can access the local system arp table.

Caution

If a device requires SNMP v3 access, you may need to install **Crypt::Rijndael** perl module. This module is not installed by default with the agent on linux systems. On a debian-based system, you just have to run `apt install libcrypt-rijndael-perl`. On a RPM-based system, running `dnf install perl-Crypt-Rijndael` is the right command.

As for any other server controlled task, the agent should use either managed or half-managed mode, as explained in *Execution modes*. If the task is server-triggered, the agent must run in managed mode, and its HTTP port should be reachable from the server.

¹ For ICMP Echo & ICMP timestamp definition, see [RFC 792](#):

- ICMP Echo messages have type 8 for requests and 0 for answers.
- ICMP Timestamp messages have type 13 for requests and 14 for answers.

1.4.2.2.2 Command-line execution

A network discovery task can be also performed without a GLPI server, allowing easier control and troubleshooting, with the *glpi-netdiscovery* command-line tool.

However, this command generates files which will have to be injected in GLPI server using *glpi-injector* command.

1.4.2.2.3 Efficiency concerns

Credentials

Unfortunately, there is no way to distinguish a failed SNMP authentication attempt on a device from the absence of a device. It means the agent will try each available credential against each IP address, in given order, and wait for timeout each time. The most efficient way to address this issue is to only use the relevant set of credentials, and reduce the specific SNMP timeout.

Threads number

In order to scan multiple addresses simultaneously, the agent can use multiple discovery threads. This allows multiple simultaneous requests, but also increases resource usage on agent host.

1.4.2.3 Advanced Configuration

Since GLPI-Agent 1.14, *glpi-agent* can use a dedicated configuration file to tune the way snmp discovery works. The file is named `snmp-advanced-support.cfg` and you should create the `snmp-advanced-support.local` which is by default included from the `.cfg` file to update the configuration.

As of 1.14, you can only change one configuration value: `oids`

`oids`

`oids` is a comma-separated list of oids used during session testing. All oids will be requested and only one has to respond to validate a snmp session. If none provides any answer, this will mean there's no device or the device is not reachable.

The default is to only check for device `sysDescr (.1.3.6.1.2.1.1.1.0)`. It means netdiscovery task will only discover by default snmp devices supporting this oid.

Fortunately, a manufacturer may decide to not support that standard oid. If this is the case, you can add a well-known supported oid to make your device discovered. As an example, this is exactly the case for Snom IP phones and you can simply add `.1.3.6.1.2.1.7526.2.4` as described in the default advanced configuration.

1.4.2.4 Troubleshooting

1. The task doesn't run at all

- The agent may be lacking the NetDiscovery module: run `glpi-agent --list-tasks` to check installed modules.
- There may be a server/agent communication issue: check you can reach the agent port (62354 by default) from the server host.
- The agent may be ignoring server requests, due to a trust issue: check the agent logs for `[http server] invalid request (untrusted address)` message.

2. The task runs, but agent logs show that SNMP is not used

- The agent may be lacking the required `Net::SNMP` perl module: run `perl -MNet::SNMP` on agent host to check, it should block.
- There may be no SNMP credentials associated to the network scanned.

3. The task runs, but no devices get added to my inventory

The reported items are insufficiently identified to be imported, according to your current import rules, check the list of ignored devices and the list of import rules on server side.

4. The task runs, but my SNMP devices are not properly identified

The agent probably lacks the device SNMP identifier in its internal database.

Use *glpi-netdiscovery* executable with debug option on the device, get the value from its output, and add it to the `sysobject.ids` file, as described in *SNMP device IDS database* to fix the issue.

```
$ glpi-netdiscovery --first 192.168.0.1 --last 192.168.0.1 --credentials version:2c,community:public
```

Output:

```
...  
[debug] partial match for sysobjectID .1.3.6.1.4.1.311.1.1.3.1.1 in database:␣  
↪ unknown device ID  
AAAAAAAAAAAAAAAA
```

5. Agent crashes

This is likely to be a TLS multithreading issue. There are multiple ways to reduce the probability of such crash:

- make sure you only have one TLS perl stack installed on the agent host, preferably `IO::Socket::SSL + Net::SSLeay`. Having both stacks at once (`IO::Socket::SSL + Net::SSLeay` vs `Net::SSL + Crypt::SSLeay`) usually leads to unexpected results, even without thread usage
- use latest upstream release of `IO::Socket::SSL`, even if your distribution doesn't provide it
- reduce threads number during network discovery tasks

However, the only actual solution currently is to disable SSL completely, using plain HTTP for agent/server communication. If the agent runs on server host, that's usually not really a problem.

1.4.3 Network inventory

A network inventory task aims to retrieve exhaustive information from SNMP compatible devices, such as network devices or printers, already part of the list of known assets.

This task can only be performed on devices already part of the list of known assets, either as a result of a previous *Network discovery task*, or manually created, with proper SNMP credentials.

1.4.3.1 Overview

This task uses SNMP to retrieve various information from a device, so as to update it in GLPI:

- consumable levels and print counter on printers
- virtual LANs definition, network topology, network ports status on network devices

1.4.3.2 Running

1.4.3.2.1 Pre-requisite

The agent performing the task needs to have the `NetInventory` module installed.

The agent performing the task needs network access to the target devices, with proper access control: just being able to send UDP packets to a device is not enough, if this device is configured to ignore them.

Caution

If a device requires SNMP v3 access, you may need to install **Crypt::Rijndael** perl module. This module is not installed by default with the agent on linux systems. On a debian-based system, you just have to run `apt install libcrypt-rijndael-perl`. On a RPM-based system, running `dnf install perl-Crypt-Rijndael` is the right command.

The target device should be associated with proper SNMP credentials in GLPI.

As for any other server controlled task, the agent should use either managed or half-managed mode, as explained in *Execution modes*. If the task is server-triggered, the agent must run in managed mode, and its HTTP port should be reachable from the server.

1.4.3.2.2 Command-line execution

A network inventory task can be also performed without a GLPI server, allowing easier control and troubleshooting, with the *glpi-netinventory* command-line tool.

However, this command generates files which will have to be injected in GLPI server using *glpi-injector* command.

1.4.3.3 Advanced Configuration

See *NetDiscovery Advanced Configuration*.

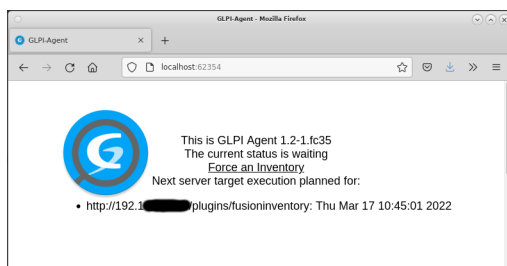
1.4.3.4 Troubleshooting

See *NetDiscovery troubleshooting*.

1.5 HTTP Interface

When run as a daemon or a service, GLPI Agent can be reached on an HTTP interface which, by default, can be reached on port 62354.

The HTTP interface, by default, provides few basic features.



It shows agent version, configured targets, next planned run date, and eventually a link to force tasks run if your ip is in the *trusted list*.

The interface provides also an API on `/status` page for GLPI server. It returns a short plain text depending on what the agent is currently doing, like:

- status: waiting
- status: running task inventory

1.5.1 Security

Regarding security, by default, this interface provides a really limited public access as it only permits to see basic informations.

Anyway, when required, the HTTP interface can be disabled by setting *no-httpd* configuration.

Listening interface can be changed using *httpd-ip* configuration to restrict it, as by default all network interfaces are used to listen for new connection. Also the port can be changed to listen on a different port using *httpd-port* configuration.

Trusted IP is limited by default and you can trust new ip or iprange configuring *httpd-trust*.

The GLPI server ip is always added to the trusted ip list.

GLPI Agent is provided with few HTTP server plugins which are all disabled by default but can extend the HTTP interface with few nice features. Regarding security, the **SSL Server plugin** can be used to encrypt HTTP exchange using SSL protocol and with GLPI Agent 1.5, the **Basic Authentication Server Plugin** can be used in combination to require authentication (See *Basic Authentication use cases*).

1.5.2 Plugins

1.5.2.1 Basic Authentication Server Plugin

By default, the agent embedded HTTP interface doesn't support authentication.

The purpose of this plugin is to enable basic authentication on the embedded HTTP interface to secure some exchanges with external clients.

Note

Basic Authentication Server Plugin is available since GLPI Agent 1.5

1.5.2.1.1 Setup

By default, this plugin is disabled. The first step is to enable it creating a dedicated configuration:

1. Locate the `basic-authentication-server-plugin.cfg` file under the GLPI agent *configuration folder*¹,
2. Make a copy of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `basic-authentication-server-plugin.local` and set `disabled` to `no`

This way, the agent will start to require authentication on few local url like `http://127.0.0.1:62354/runnow` and `http://127.0.0.1:62354/status`.

1.5.2.1.2 Configuration

The default configuration is self-explanatory:

```
# By default, a plugin is always disabled unless "disabled" is set to "no" or "0".
# You can uncomment the following line or set it in included configuration file
# at the end of this configuration
#disabled = no

# Set url matching regexp to enable basic authentication on, default to any
# Could be set to /toolbox/. * to enable authentication only on ToolBox plugin
#url_path_regexp = .*
```

(continues on next page)

¹ On windows, the configuration is also a file and it located under the `etc` sub-folder of the GLPI Agent installation folder.

(continued from previous page)

```

# Port on which to request authentication for anonymous requests, default to legacy port
#port = 62354

# user and password to be used for authentication, empty by default, must be set to
# enable the plugin
#user = [string without a ':' char]
#password = [string]

# A realm to be presented to http client, default to "GLPI Agent"
#realm = GLPI Agent

# To limit any abuse we expect a maximum of 600 requests by 10 minutes (600 seconds)
# You can adjust the rate limitation by updating the requests number limit
# or the period on which apply the requests number limit
#maxrate          = 600
#maxrate_period = 600

# You should create and define you specific parameter in the following
# included configuration file to override any default.
# For example just set "disabled = no" in it to enable the plugin
include "basic-authentication-server-plugin.local"

```

disabled

Can be set to "no" to enable the plugin. (By default: yes)

url_path_regexp

A regexp defining on which full path must match to support authentication on. (By default: .*)

For example, you can enable authentication only on Toolbox by setting it to /toolbox.*.

port

Can be set to the port on which you need to enable authentication. (By default: 62354)

You can for example keep simple http support on the default port and just enable authentication on the port used by another server plugin.

user

The user name to be used by client to validate authentication. (By default: not defined)

password

The user password to be used by client to validate authentication. (By default: not defined)

realm

A string for basic authentication required by integrated http server. (By default: **Glpi Agent**)

It defines a site zone where the authentication applies and may appear in the authentication form.

maxrate and maxrate_period

Limit requests for a given ip to **maxrate** other the **maxrate_period** time (in seconds). (By default: **600** requests by **600** seconds for a single ip)

This 2 parameters could be used to limit abuse if the agent is listening on a public network.

1.5.2.1.3 Basic Authentication use cases

Enabling SSL and basic authentication for ToolBox

As *ToolBox interface* can show sensible information and can be used to control few tasks, you may want to secure it by:

1. Enable *ToolBox interface* on a dedicated port,
2. Enable this basic authentication plugin on that same port for all paths by just setting a port, a user and a password
3. Enable ssl plugin on that same port so every communication with the agent are encrypted, including the basic authentication challenge which can expose user and password to network otherwise.

For example, enable the *ToolBox interface* with such `toolbox-plugin.local` configuration:

```
disabled = no
port = 8888
```

Then, enable the basic authentication with such `basic-authentication-plugin.local` configuration:

```
disabled = no
port = 8888
user = admin
password = mystrongpassword
```

And finally, enable the *SSL Server Plugin* with such `ssl-server-plugin.local` configuration:

```
disabled = no
ports = 8888
# openssl req -x509 -newkey rsa:2048 -keyout etc/key.pem -out etc/cert.pem -days 3650 -
↪sha256 -nodes -subj "/CN=A.B.C.D"
ssl_cert_file = cert.pem
ssl_key_file = key.pem
```

`cert.pem` and `key.pm` can be generated by the following command on linux:

```
$ openssl req -x509 -newkey rsa:2048 -keyout etc/key.pem -out etc/cert.pem -days 3650 -sha256 -nodes
```

Then you'll be able to access your *ToolBox interface* with `https://A.B.C.D/toolbox` URL. There you would have to trust the server certificate and then log in with `admin:mystrongpassword` as credential. The `https://admin:mystrongpassword@A.B.C.D/toolbox` URL can also be used.

1.5.2.2 SSL Server Plugin

By default, the agent embedded HTTP interface doesn't support message encryption using SSL.

The purpose of this plugin is to enable SSL on the embedded HTTP interface to secure all exchanges with external clients.

1.5.2.2.1 Setup

By default, this plugin is disabled. The first step is to enable it creating a dedicated configuration:

1. Locate the `ssl-server-plugin.cfg` file under the GLPI agent *configuration folder*¹,
2. Make a copy of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `ssl-server-plugin.local` and set `disabled` to `no`

¹ On windows, the configuration is also a file and it located under the `etc` sub-folder of the GLPI Agent installation folder.

This way, the agent will start to only accept client supporting SSL. For instance, if you accessed before the agent interface on local machine using `http://127.0.0.1:62354`, you'll now have to use `https://127.0.0.1:62354`.

1.5.2.2.2 Configuration

The default configuration is self-explanatory:

```
# By default, a plugin is always disabled unless "disabled" is set to "no" or "0".
# You can uncomment the following line or set it in included configuration file
# at the end of this configuration
#disabled = no

# Comma separated list of ports like in: ports = 62355,62356
#ports = 0

# Example command to generate key/certificate files pair
# openssl req -x509 -newkey rsa:2048 -keyout etc/key.pem -out etc/cert.pem -days 3650 -
# sha256 -nodes -subj "/CN=127.0.0.1"
#ssl_cert_file = cert.pem
#ssl_key_file = key.pem

# You should create and define you specific parameter in the following
# included configuration file to override any default.
# For example just set "disabled = no" in it to enable the plugin
include "ssl-server-plugin.local"
```

disabled

Can be set to "no" to enable the plugin. (By default: yes)

ports

Can be set to a list of ports on which you need to enable SSL support. (By default: 0)

You can for example keep simple http support on the default port and just enable SSL on the port used by one or more agent server plugins.

ssl_cert_file

The path to the server certificate to use with SSL support. (By default: not defined)

The path can be relative to the configuration folder or an absolute path.

ssl_key_file

The path to the server private key certificate to use with SSL support. (By default: not defined)

The path can be relative to the configuration folder or an absolute path. This path should be a secured location, not readable by simple local system users.

Note

ssl_cipher option is available since GLPI Agent v1.2

ssl_cipher

Set cipher to use for SSL support or set ciphers to disable. (By default: not defined)

You can for TLS v1.3 support by setting: `ssl_cipher = TLSv13`

You disable SSL v3 and TLS v1.1 support by setting: `ssl_cipher = !SSLv3:!TLSv11`

1.5.2.3 Proxy Server Plugin

The purpose of this plugin is to enable a proxy mode on the embedded HTTP interface.

Fig. 1: Proxy Server Plugin diagram

It can replace a proxy pass configuration on a http server or a complex proxy setup. It offers few other advantages and features that can be used in advanced setup.

1.5.2.3.1 Setup

By default, this plugin is disabled. The first step is to enable it creating a dedicated configuration:

1. Locate the `proxy-server-plugin.cfg` file under the GLPI agent *configuration folder*¹,
2. Make a copy of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `proxy-server-plugin.local` and set `disabled` to `no`

This way, the agent will start to accept inventory submission on its current port and on `/proxy/glpi` as base url. If a GLPI server is setup, any submitted inventory will be forwarded to that GLPI server.

Note

You can also enable a **secondary proxy** by creating the `proxy2-server-plugin.local` configuration. The purpose here is to enable another proxy configuration on the same agent, mostly in the case you need it on a dedicated port with eventually SSL support plugin enabled.

1.5.2.3.2 Configuration

The default configuration is self-explanatory:

```
# By default, a plugin is always disabled unless "disabled" is set to "no" or "0".
# You can uncomment the following line or set it in included configuration file
# at the end of this configuration
#disabled = no

# Set base url matching for API
#url_path = /proxy
# Note: the server URL to set on client would have to be http[s]://[host]:[port][url_
↪path]/glpi
# By default, this should be: http://[agent-ip-or-dns]:62354/proxy/glpi

# Port on which to listen for inventory requests, default to legacy port
#port = 62354

# The delay the proxy should return as contact timeout to agents (in hours)
#prolog_freq = 24

# Option to handle proxy local storing. Set a folder full path as local_store to
# also store received XML locally. Set only_local_store to not immediatly send
# received XML to known server(s).
#only_local_store = no
```

(continues on next page)

¹ on windows the configuration is also a file under the `etc` sub-folder of the GLPI Agent installation folder.

(continued from previous page)

```
#local_store =

# To limit any abuse we expect a maximum of 30 requests by hour and by ip (3600 seconds)
# You can adjust the rate limitation by updating the requests number limit
# or the period on which apply the requests number limit
#maxrate          = 30
#maxrate_period = 3600

# The maximum number of forked handled request
#max_proxy_threads = 10

# The maximum number of proxy a request can pass-through
#max_pass_through = 5

# By default, if a GLPI server is set, we consider it supports GLPI protocol
# otherwise this proxy should only support legacy XML based protocol
#glpi_protocol = yes

# no-category config returned to agent when using CONTACT protocol without GLPI server
# or if only_local_store is set to yes
# no_category =
# Example: no_category = process,environment

# You should create and define you specific parameter in the following
# included configuration file to override any default.
# For example just set "disabled = no" in it to enable the plugin
include "proxy-server-plugin.local"
```

disabled

Can be set to "no" to enable the plugin. (By default: yes)

port

Can be set to a port on which the agent will listen too. (By default: 0, meaning use agent port)

You can dedicate a port for the proxy usage. You can even enable the *SSL Server Plugin*, and set the port in its ports list to force using SSL with the proxy mode.

prolog_freq

In the case the proxy agent is not in touch with a GLPI server, this is the delay time in hours an inventory should be sent by contacting agents. (By default: 24)

only_local_store

Set to yes to only store locally submitted inventories. (By default: no)

This can be handy if the only purpose is to collect inventories and no GLPI server is reachable. Stored inventories could then be passed later to *glpi-injector*.

local_store

This is a full path where to store submitted inventories. If set, the proxy agent will always stored submitted inventories. (By default: empty)

You must manage by yourself the stored inventories or you may face a storage outage after a while if many agents submits inventories. But as inventories are stored with the deviceid as file basenane, new inventory for a known agent will just replace the existing one.

This storage may be used as an inventory backup solution but keep in mind this storage should be keep secured as it will contains a lot of sensible datas.

maxrate and maxrate_period

Limit requests for a given ip to **maxrate** other the **maxrate_period** time (in seconds). (By default: **30** requests by **3600** seconds for a single ip)

This 2 parameters could be used to limit abuse if the agent proxy if listening on a public network.

By default the average request for a given ip should be lower than 2. But this can be greater if the other ip is a chained proxy. In that last case, you may need to grow that values.

max_proxy_threads

This set the maximum number of single requests the proxy agent can handle at the same time. (By default: 10)

max_pass_through

This set the maximum number of proxy agents a single inventory submission can pass. (By default: 5)

Each time a inventory is submitted, a HTTP header value is set or increased. If that value reaches the **max_pass_through** of a proxy agent, the inventory won't be submitted to the next proxy agent.

Changing this parameter is only needed when you're using a chained proxy agents configuration and when you have at least 5 proxy agents in the chain. This parameter is a security to block loops in the case of chained proxy agents misconfiguration pointing to each others.

glpi_protocol

Set to "no" if you don't want to use new GLPI Agent Protocol needed for GLPI native inventory. (By default: yes)

When set to "no", the proxy agent will just act as a legacy GLPI server supporting XML inventory format only. Otherwise, it will tell remote agent to use new protocol which involves to send inventory in JSON format.

no_category

This is a comma separated list of category to not include in inventories. It has the same purpose than *no-category configuration* but set on server-side. This only works with new GLPI Agent Protocol and JSON format.

1.5.2.3.3 Use cases

Private network inventory storage

In the case, you have a private network from where no device can access the GLPI server, you can:

First, store submitted inventory using a proxy agent with such `proxy-server-plugin.local` configuration:

```
disabled = no
only_local_store = yes
local_store = /var/glpi-agent/proxy
```

Then, from GLPI server or a dedicated platform with GLPI server access possible, get a copy of stored inventories into a dedicated folder:

- using a command like `scp`, `ftp` or `rsync`
- using an USB key

Finally, inject inventories into GLPI with *glpi-injector* script:

```
$ glpi-injector -d /var/glpi-agent -r -R -u http://my-glpi-server/
```

Proxy with HTTP and HTTPS support

In the case you need to secure a private network with eventually old FusionInventory agent not supporting SSL, but you still need SSL for newer GLPI Agents, you can create a proxy agent listening on port 80 and port 443.

First, enable main proxy mode on port 443 with the following `proxy-server-plugin.local` configuration:

```
disabled = no
port = 443
```

Secondly, enable the secondary proxy mode on port 80 with the following `proxy2-server-plugin.local` configuration:

```
disabled = no
port = 80
```

Also enable the *SSL Server Plugin* with the following `ssl-server-plugin.local` configuration:

```
disabled = no
ports = 443
```

Now, in agent configuration having access to the proxy agent, you can use any one of the 2 following URL as *server configuration*:

```
server = http://proxy-agent/proxy/glpi
server = https://proxy-agent/proxy/glpi
```

Here you don't have to specify the port as standard http and https ports are used. The only requirement for the proxy agent is to run on a dedicated server with that ports not used by any other service.

Chained proxies

Imagine you want to inventory the devices from one factory of your company in a given town having its dedicated and private network. This factory network is only visible via a vpn at the town level through a network being able to see other factories in the same town. Now your GLPI server is located in your head quarter in another town linked through a dedicated network link.

You can first create a proxy agent at the factory level just by enabling the proxy plugin on one computer, let's say the one with 10.77.200.55 ip. You just need to enable the plugin with the following `proxy-server-plugin.local` configuration:

```
disabled = no
```

Do the same on other agent where you need the plugin, the one at the town level, let's say it has the 10.77.0.2 ip. And do also the same with the agent in the head quarter network with let's say the 10.1.0.120 ip.

On all other devices in the factory, setup agents with the following URL as *server configuration*:

```
server = http://10.77.200.55:62354/proxy/glpi
```

On the agent with the proxy plugin enabled, set the server URL to the proxy agent enabled at the town level:

```
server = http://10.77.0.2:62354/proxy/glpi
```

On the agent at the town level with the proxy plugin enabled, set the server URL to the proxy agent enabled at the head quarter level:

```
server = http://10.1.0.120:62354/proxy/glpi
```

On the agent at the head quarter level, just set the normal GLPI server URL as *server configuration*.

Then you just have to secure your network to permit each proxy agent to communicate on port 62354 with its chained one.

Note

Remember only 5 proxy agents can be chained by default (see *max_pass_through parameter*). If you want to chain more proxy agents, set this parameter value to a higher value starting from the 6th proxy agent.

1.5.2.4 Inventory Server Plugin

This plugin purpose is to permit requesting an inventory from a trusted remote computer knowing a shared secret. This is handy in the case the agent has no way to contact the GLPI server by itself.

Fig. 2: Inventory Server Plugin diagram

1.5.2.4.1 Setup

By default, this plugin is disabled. The first step is to enable it creating a dedicated configuration:

1. Locate the `inventory-server-plugin.cfg` file under the GLPI agent *configuration folder*¹,
2. Make a copy of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `inventory-server-plugin.local` and set `disabled` to `no`
4. Also set a private token setting the `token` parameter to any strong secret

This way, the agent will start to accept inventory requests from a computer knowing the shared secret. To make such inventory requests, you'll just have to use the **agent** sub-command of *glpi-remote* script.

Hint

You can also enable the *SSL Server Plugin* to also encrypt the communication.

Note

This HTTP server plugin feature has nothing to do with the glpi-agent remote inventory feature as here, you still need to install a GLPI Agent on the targeted systems. With the glpi-agent remote inventory feature, you don't need any agent on the targeted systems.

1.5.2.4.2 Configuration

The default configuration is self-explanatory:

```
# By default, a plugin is always disabled unless "disabled" is set to "no" or "0".
# You can uncomment the following line or set it in included configuration file
# at the end of this configuration
#disabled = no

# Set base url matching for API
#url_path = /inventory

# Port on which to listen for inventory requests, default to legacy port
```

(continues on next page)

¹ on windows the configuration is also a file under the `etc` sub-folder of the GLPI Agent installation folder.

(continued from previous page)

```
#port = 62354

# token as private secret used to verify authorization token, not defined by
# default, but mandatory to access API
#token = [secret-string]

# A timeout for session to no more trust a client payload
#session_timeout = 60

# Set this to 'yes' if XML compression is not required
#no_compress = no

# To limit any abuse we expect a maximum of 50 requests by hour (3600 seconds)
# You can adjust the rate limitation by updating the requests number limit
# or the period on which apply the requests number limit
#maxrate           = 30
#maxrate_period    = 3600

# You should create and define you specific parameter in the following
# included configuration file to override any default.
# For example just set "disabled = no" in it to enable the plugin
include "inventory-server-plugin.local"
```

disabled

Can be set to "no" to enable the plugin. (By default: yes)

url_path

The path to the server certificate to use with SSL support. (By default: **/inventory**)

The path can be relative to the configuration folder or an absolute path.

port

Can be set to a port on which the agent will listen too. (By default: **0**, meaning use agent port)

You can dedicate a port for the inventory usage. You can even enable the *SSL Server Plugin*, and set the port in its ports list to force using SSL with the inventory plugin.

token

MUST be set to a strong secret or no inventory will be generated. (By default: not defined)

session_timeout

The session timeout is a time in seconds and defines the maximum time the agent will wait for the remote client to authenticate itself with the shared secret. (By default: **60**)

no_compress

Can be set to **yes** to avoid inventory compression when sent back. (By default: **no**)

maxrate and maxrate_period

Limit requests for a given ip to **maxrate** other the **maxrate_period** time (in seconds). (By default: **30** requests by **3600** seconds for a single ip)

This 2 parameters could be used to limit even more any brute force attack attempt.

1.5.2.4.3 Inventory request

Inventory request have to be done using **agent** sub-command of the `glpi-remote` script.

See the *glpi-remote* dedicated man page for all possible options.

1.5.2.4.4 Use cases

DMZ server inventory

In the case you have a server in DMZ which cannot access the GLPI server, but the GLPI server is authorized to reach it. You still can install an agent on GLPI server to request remotely inventory to a listening remote agent.

Then first, enable the plugin with such `inventory-server-plugin.local` configuration:

```
disabled = no
token = 5c9898f9-e619-4bdb-8e29-6a20766ab760
```

In the agent conf, don't set `server` nor `local` but set `listen` to `yes` and set `httpd-trust` with the GLPI server one. For example create the `/etc/glpi-agent/conf.d/local.cfg` file with:

```
listen = yes
httpd-trust = <glpi-server-ip>
```

On the GLPI server, create a script you would want to put in `/etc/cron.daily`:

```
#!/bin/bash
sleep $((RANDOM/100))
glpi-remote -T 5c9898f9-e619-4bdb-8e29-6a20766ab760 agent <dmz-server-ip> | \
  glpi-injector -url http://127.0.0.1/ >/var/tmp/server-inventory.log 2>&1
```

Adapt this shell script to your needs.

Internet server

In the case you have an internet server hosted anywhere and you want to inventory it in your GLPI being in your intranet.

Make sure server and intranet firewalls will permits communications between them, the GLPI server being the HTTP client and let's say via the 54443 port.

Then first, after installed the agent on the internet server, enable the plugin with such `inventory-server-plugin.local` configuration:

```
disabled = no
token = 2b0a48a2-6eb1-4e8f-bf8c-41f461b58ef1
base = /2cd3a12ac1c4
port = 54443
```

Also enable the *SSL Server Plugin* with such `ssl-server-plugin.local` configuration:

```
disabled = no
ports = 54443
```

In the agent conf, don't set `server` nor `local` but set `listen` to `yes` and set `httpd-trust` with your intranet public one. For example create the `/etc/glpi-agent/conf.d/glpi.cfg` file with:

```
listen = yes
httpd-trust = <intranet-public-ip>
```

On the GLPI server, create a script you would want to put in `/etc/cron.daily`:

```
#!/bin/bash
sleep $((RANDOM/100))
glpi-remote -T 2b0a48a2-6eb1-4e8f-bf8c-41f461b58ef1 -p 54443 --ssl --no-ssl-check -b /
2cd3a12ac1c4 agent <internet-server-ip> | \
glpi-injector -url http://127.0.0.1/ >/var/tmp/internet-server-inventory.log 2>&1
```

Adapt this shell script to your needs.

1.5.2.5 Test Server Plugin

This plugin has indeed no functional purpose but it can be used as a template if you need to develop your own GLPI Agent HTTP server plugin.

This plugin when enabled only log the request and just answer to it with a HTTP 200 return code if its URL matched the `^/test/([\w\d/-]+)?$` regular expression.

See the [code](#) for more details.

1.5.2.5.1 Setup

By default, this plugin is disabled. The first step is to enable it creating a dedicated configuration:

1. Locate the `server-test-plugin.cfg` file under the GLPI agent [configuration folder](#)¹,
2. Make a copy of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `server-test-plugin.local` and set `disabled` to `no`

1.5.2.5.2 Configuration

The default configuration is:

```
# By default, a plugin is always disabled unless "disabled" is set to "no" or "0".
# You can uncomment the following line or set it in included configuration file
# at the end of this configuration
#disabled = no

#configtest = test

#port = 62355

# You should create and define you specific parameter in the following
# included configuration file to override any default.
# For example just set "disabled = no" in it to enable the plugin
include "server-test-plugin.local"
```

disabled

Can be set to "no" to enable the plugin. (By default: yes)

port

Can be set to a port on which the agent will listen too. (By default: 0, meaning use agent port)

¹ on windows the configuration is also a file under the `etc` sub-folder of the GLPI Agent installation folder.

You can dedicate a port and even enable the *SSL Server Plugin* setting this port in its `ports` list to force using SSL with this Test plugin.

configtest

Just a value that would be logged with the request log line.

1.5.2.6 ToolBox interface

ToolBox is a simple web interface embedded into GLPI Agent that allows users to configure some features when there is no GLPI server available.

For example:

- When you have an isolated network that cannot or should not be connected to a GLPI Server or via GLPI Agent *Proxy Server Plugin* for any reason but you still need to run a network discovery or a network inventory.
- When you need to review and update the inventory with custom fields which can be setup using a YAML file exported from *GLPI Fields plugin*.

Here are the main features **ToolBox** provides:

- manage inventory task jobs (available since GLPI Agent 1.6)
- manage credentials definitions
- manage ip range definitions
- manage scheduling definitions (available since GLPI Agent 1.6)
- display light inventory results and eventually edit custom fields
- manage remotes definition (deprecated since GLPI Agent 1.6)
- manage mibsupport rules to tune the results for SNMP network devices

Note

ToolBox is not intended to replace a plugin like *GlpInventory* or *FusionInventory* for GLPI plugins but can be helpful where none of these plugin can or should be used for any reason.

See also

Don't miss the [Unlocking Efficient Network Inventory Management with the GLPI Agent Toolbox Plugin](#) blog post including this demo video by Arthur Schaefer:

<https://youtu.be/xNfqKTP9LN8>

Again **ToolBox** is mainly a GLPI-Agent based tool allowing to run a netscan tasks over a network only using a glpi-agent and a browser. Netscan task is firstly used to discover and inventory networks devices or printer supporting SNMP, but now (available since GLPI Agent 1.6), it also supports to discover and inventory computers like ESX, Unix/Linux supporting ssh and windows computers supporting WinRM using *RemoteInventory task*.

Run tasks results can be stored in the agent environment and you can browse them on **Results** dedicated pages. But (available since GLPI Agent 1.6) you can also simply and directly send them to a GLPI server, without storing them locally. You can indeed select as job target any glpi-agent configured target (local or server) or the local glpi-agent installation folder.

Note

Since GLPI Agent 1.5, **ToolBox** also permits to manage *RemoteInventory task* by defining and updating remotes. It can also be used to expire remotes and require a remoteinventory task run to handle any expired remotes right now. The interface is really basic: it doesn't show the status of a remote and you'll still have to audit your agent log if some remotes are failing to upload an inventory in GLPI.

For this reason, and as GLPI Agent 1.6 will provide a netscan feature supporting RemoteInventory task, the remotes page is **deprecated** and will be *removed* in a future version.

1.5.2.6.1 Setup

By default, **this plugin is disabled**. So the first required step is to enable it creating a dedicated configuration:

1. Locate the `toolbox-plugin.cfg` file under the GLPI agent *configuration folder*¹,
2. **Make a copy** - avoid renaming it - of this file in the same folder by just changing the file extension from `.cfg` to `.local`.
3. Edit the `toolbox-plugin.local` and set `disabled` to `no` and comment or remove the `include` directive at the end
4. Restart the `glpi-agent` service

This way, the agent will start to accept toolbox requests on its current port and on `/toolbox` as base url, by default: `http://127.0.0.1:62354/toolbox`

Warning

By default, anybody can access this feature after it has been enabled. You should first set `forbid_not_trusted = yes` in your `toolbox-plugin.local` to enable a "by trusted IP address" filtering, authorizing IP only enabled with the *htp-trust* option.

You **MUST** not enable **ToolBox** on an unsecure network.

Also, **since GLPI Agent 1.5**, you can enable *Basic Authentication Server Plugin* and *SSL Server Plugin* to completely secure the **ToolBox** interface.

By default, you have a very restricted interface displayed on the first access. But you can edit everything under the setup page clicking on the top right gear icon. That is where you can activate additional pages in the *Navigation* bar section.

You should enable the following pages:

1. *Credentials*: to manage SNMP credentials, ESX user/password, SSH and WinRM credentials for RemoteInventory netscan
2. *Inventory*: to configure and manage jobs for local or netscan inventory tasks
3. *IP Ranges*: to manage ip ranges and to define credentials to use on each one
4. *Scheduling*: to define scheduling to be used by inventory jobs

You can disable the **Results** page if your `glpi-agent` will directly submit inventories to a GLPI server and you won't use the local agent installation folder as target.

¹ on windows the configuration is also a file under the `etc` sub-folder of the GLPI Agent installation folder.

Note

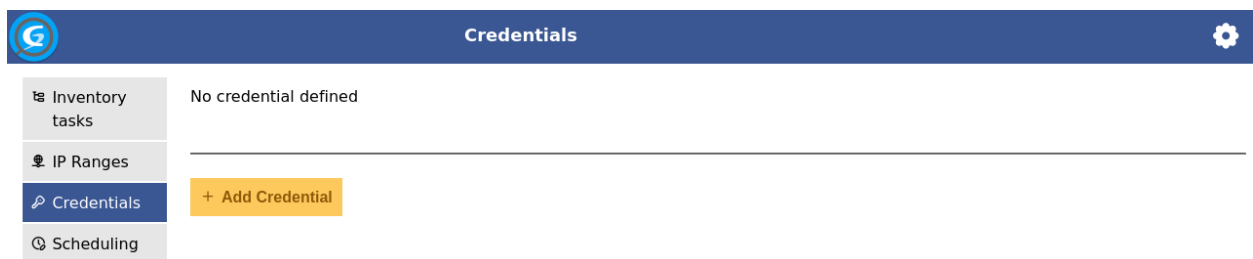
After you have configured your interface, you can disable any further online configuration to avoid mistake by disabling the Configuration update authorized checkbox in the Toolbox plugin configuration section.

If you need to tune again the configuration, you need to edit the toolbox.yaml file and change the updating_support line in the configuration section like:

```
configuration:
  updating_support: yes
```

1.5.2.6.2 Credentials

Initially, the credentials page will show you it found no credential:



So it essentially gives you access to the Add Credential button.

Create a credential

You can quickly create a new credential after you have clicked on the Add Credential button on the Credentials list page:

You have then a simple form permitting you to set a **Name**, choose a **Type** and when applicable:

1. Set the SNMP version

2. Set the SNMP community
3. Set the SNMP port if different than the default 161
4. Set the SNMP protocol, the default being *udp*
5. Set the SNMP username for SNMP v3
6. Set the SNMP authentication password for SNMP v3
7. Set the SNMP authentication protocol for SNMP v3
8. Set the SNMP privacy password for SNMP v3
9. Set the SNMP privacy protocol for SNMP v3
10. Set the username for a remote credential (ssh, winrm or esx)
11. Set the authentication password for a remote credential
12. Set the remote authentication port if different than the defaults: 22 for ssh, 5985 for winrm or 5986 for winrm with ssl mode enabled
13. Enable one or more remote inventory mode for ssh or winrm

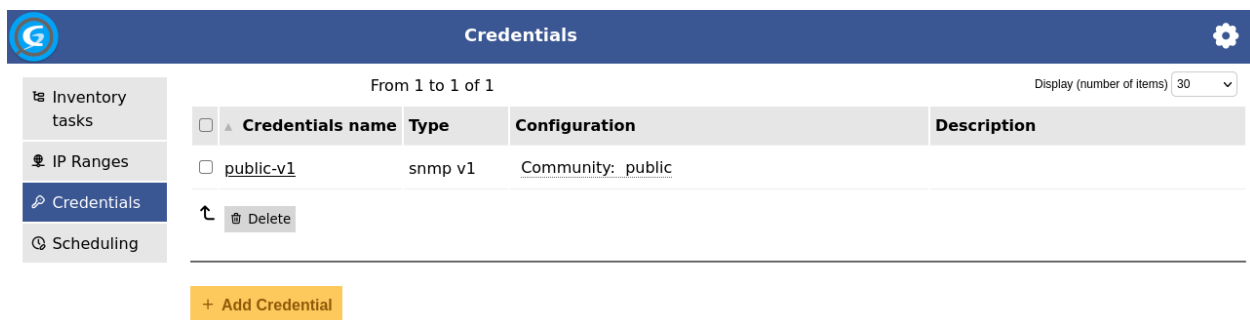
You can also define a description for this credentials if this can help you to manage them. It is not used by GLPI Agent and it's up to you to use it.

The **name** is free but **mandatory** and will be used as key name to associate it to IP ranges. So choose it carefully to be meaningful for your credentials management.

Note

Passwords are not shown but you have an eye icon on right of the field to click on if you need to check them.

When you click on **Create Credential**, the agent will check few field and will add it to the list unless something is wrong:



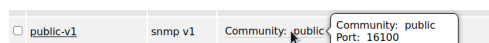
The screenshot shows the 'Credentials' management page. On the left sidebar, 'Credentials' is selected. The main content area displays a table with the following data:

Credentials name	Type	Configuration	Description
☐ public-v1	snmp v1	Community: public	

Below the table, there is a 'Delete' button and a '+ Add Credential' button.

From the credentials list, you'll always have the option to edit or delete a credential.

You also can move you mouse pointer other the config column to check few details. Passwords won't be shown:



This close-up shows the 'public-v1' row. The 'Community: public' field is highlighted, and a tooltip is visible showing the full configuration: 'Community: public' and 'Port: 16100'.

Update a credential

To update a credential, you simply can click on the **Credential** name in the **Credentials** list page:

Inventory tasks
IP Ranges
Credentials
Scheduling

Edit « public-v1 » credential

Name

Type
☒ snmp ☐ ssh ☐ winrm ☐ esx

Version
☒ v1 ☐ v2c ☐ v3

Community

Port

Protocol

Description

You obtain the same form as for *creation*. And from here, you can:

1. Rename the credential
2. Update any credential setting
3. Click on Update to save your changes
4. Click on Go back to list or on Credentials in the navigation bar to return to the list.

Delete a credential

For credential deletion, from the Credentials list, you have to click on the related checkbox, and click on the Delete button.

Warning

Deletion will be **forbidden** in the case a credential is still in use. If you really need to remove a credential, first remove it from all associated IP ranges.

1.5.2.6.3 IP Ranges

Initially, the IP ranges page will show you it found no IP range:


IP Ranges

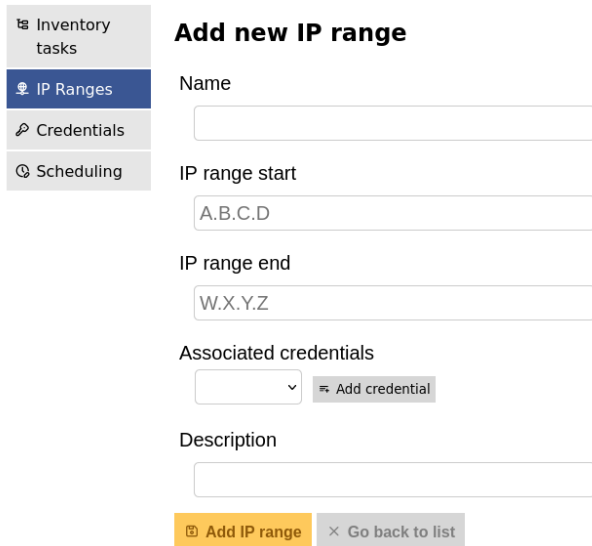

Inventory tasks
IP Ranges
Credentials
Scheduling

No IP range defined

So it essentially gives you access to the Add new IP range button.

Create an IP range

You can quickly create a new IP range after you have clicked on the Add new IP range button on the IP range list page:



Add new IP range

Name

IP range start

IP range end

Associated credentials

Description

You have then a simple form permitting you to first set:

1. the IP range **Name**
2. the **IP range start**
3. the **IP range end**

These fields are all **mandatory** to define an IP range.

The **name** format is free and will be used as a key name to associate it to an inventory job. So choose it carefully to be meaningful for you.

You can also define a description for this IP range if this can help you to manage them. It is not used by GLPI Agent and it's up to you to use it.

Note

If you only need to scan one IP, just use this ip as first and end ip of the range.

Also you should use an explicit name which will permit you to identify this ip range as targetting only one IP.

You would like also to associate one or more credentials to this new IP range. In **ToolBox**, an IP range without at least one credential will be useless during nmapscan, so you should at least *have created a first credential* before.

When you click on Add IP range, the agent will check few fields and will add it to the list unless something is wrong:

IP Ranges

From 1 to 1 of 1

Display (number of items) 30

☐	IP range name	First ip	Last ip	Credentials	Description
☐	My SNMP printer	192.168.1.122	192.168.1.122	public-v1	

Associated credentials:

From the IP Ranges list, you'll always have the option to edit or delete an IP range. But you'll also have a mass action to add or remove one credential to your IP ranges. This is handy when you want to quickly update a lot of IP ranges.

You also can move you mouse pointer other the credentials column to check related associated credential details. Passwords won't be shown:

☐	IP range name	First ip	Last ip	Credentials	Description
☐	My SNMP printer	192.168.1.122	192.168.1.122	public-v1	

Associated credentials:

Type: snmp
 SNMP version: v1
 Community: public

Update an IP range

To update an IP range, you simply can click on the IP range name in the **IP Ranges** list page:

Edit « My SNMP printer » ip range

Name

My SNMP printer

IP range start

192.168.1.122

IP range end

192.168.1.122

Associated credentials

☒ public-v1

Description

You obtain the same form as for *ip range creation*. And from here, you can:

1. Rename the IP range
2. Change the start and the end of the IP range
3. Unselect any associated credential
4. Associate another credential, only if another credential is available
5. Click on Update to save your changes
6. Click on Go back to list or on IP Ranges in the navigation bar to return to the list.

Delete an IP range

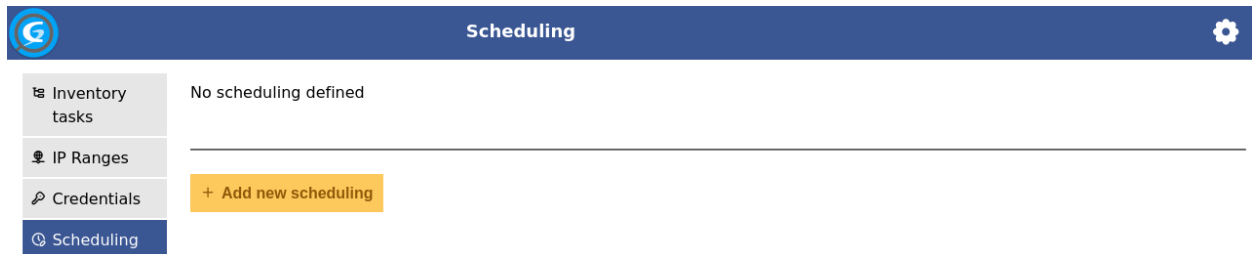
For IP range deletion, from the IP Ranges list, you have to click on the related checkbox, and click on the Delete button.

Warning

Deletion will be **forbidden** in the case an IP range is still in use. If you really need to remove an IP range, first remove it from all associated netscan inventory tasks.

1.5.2.6.4 Scheduling

Initially, the scheduling page will show you it found no scheduling:



So it essentially gives you access to the Add new scheduling button.

Create a scheduling

You can quickly create a new scheduling after you have clicked on the Add new scheduling button on the Scheduling list page:

Inventory tasks
IP Ranges
Credentials
Scheduling

Add new scheduling

Name

Type

☒ delay ☐ timeslot

Configuration

The approximate delay time between two tasks runs

Delay Time unit

24 hour

Description

Add Go back to list

You have then a simple form permitting you to first set:

1. the scheduling **Name**
2. the **Type**
3. the *delay configuration* or *timeslot configuration*

The name remains **mandatory** to define a scheduling.

The **name** format is free and will be used as a key name to associate it to an inventory job. So choose it carefully to be meaningful for you.

You can also define a description for this scheduling if this can help you to manage them. It is not used by GLPI Agent and it's up to you to use it.

When you click on Add, the agent will check few fields and will add it to the list unless something is wrong:

Scheduling

Inventory tasks

IP Ranges

Credentials

Scheduling

From 1 to 2 of 2

Display (number of items) 30

Scheduling name

Type

Configuration

Description

Every 24 hours

delay

24 hours

Every day morning

timeslot

day: all, start: 06h00, duration: 02h00

Delete

+ Add new scheduling

From the Scheduling list, you'll always have the option to edit or delete a scheduling. The scheduling details are shown in the configuration column.

Create a delay scheduling

When you *create a scheduling*, you have to choose the **delay** type. You can then configure the delay choosing a number for the delay and select a time unit from the given list:

Configuration

The approximate delay time between two tasks runs

Delay Time unit

Description

Create a timeslot scheduling

When you *create a scheduling*, you have to choose the **timeslot** type. You can then configure the timeslot choosing a week day or **all**, a day time start hour and minute, a duration number and a time unit for the duration to choose between **minute** or **hour**:

Type

☐ delay ☒ timeslot

Configuration

Week day, day time and duration of a time slot during which a task will be executed once

Week day Start Duration Time unit

Update a scheduling

To update a scheduling, you simply can click on the Scheduling name in the **Scheduling** list page:

Inventory tasks

IP Ranges

Credentials

Scheduling

Edit « Every 24 hours » scheduling

Name

Type

☒ delay ☐ timeslot

Configuration

The approximate delay time between two tasks runs

Delay Time unit

Description

You obtain the same form as for *scheduling creation*. And from here, you can:

1. Rename a scheduling
2. Update the delay duration or the timeslot configuration
3. Click on Update to save your changes
4. Click on Go back to list or on Scheduling in the navigation bar to return to the list.

Delete a scheduling

For scheduling deletion, from the Scheduling list, you have to click on the related checkbox, and click on the Delete button.

Warning

Deletion will be **forbidden** in the case a scheduling is still in use. If you really need to remove a scheduling, first remove it from all associated inventory tasks.

1.5.2.6.5 Inventory tasks

Initially, the inventory page will show you it found no inventory task:



So it essentially gives you access to the Add new inventory task button.

Create an inventory task

You can quickly create a new inventory task after you have clicked on the Add new inventory task button on the Inventory tasks list page:

Inventory tasks
IP Ranges
Credentials
Scheduling

Add new inventory task

Name

Type

☐ Local inventory
☒ Network scan

Target

Scheduling type

☒ Delay
☐ Timeslot

Delay

Use a tag

(i) Only for remote inventory and ESX tasks

[Add new tag](#)

Associated ip ranges

Threads

Connection timeout

Description

You have then a simple form permitting you to first set:

1. the inventory task **Name**
2. the **Type** between **Local inventory** and **Network scan**
3. a target to choose between **Agent folder** and the configured ones via **server** or **local** parameters
4. a scheduling type
5. a delay or one or more timeslots depending on the choosen scheduling type
6. an optionnal tag to use on computers inventory
7. an IP range to associate if the task type is **Network scan**
8. a threads count if the task type is **Network scan** and to parallelize the task run
9. a connection timeout to use during **Network scan**

The name remains **mandatory** to define an inventory task.

The **name** format is free but choose it carefully to be meaningful for your tasks management.

You can also define a description for this inventory task if this can help you to manage them. It is not used by GLPI Agent and it's up to you to use it.

Warning

Local inventory task should not be configured via **ToolBox** as this remains the first GLPI Agent role when **inventory** task is not disabled in agent configuration.

This inventory task type should probably only be used for tests or when you need to run it manually from a portable installation, for example using an usb disk on a computer in an isolated network.

When you click on **Create inventory task**, the agent will check fields and will add it to the list unless something is wrong:

The screenshot shows the 'Inventory tasks' interface. On the left is a sidebar with 'Inventory tasks' selected. The main area displays a table with one task: 'Scan my printer' (Type: Network scan, Scheduling: Every 24 hours, Next run date: Disabled task). The configuration column shows: Target: server0, IP range: My SNMP printer, Threads: 1, Timeout: 1s. Below the table are buttons: Run task, Enable, Disable, and Delete. At the bottom is a '+ Add new inventory task' button.

Task name	Type	Scheduling	Last run date	Next run date	Configuration	Description
Scan my printer	Network scan	Every 24 hours		Disabled task	Target: server0 IP range: My SNMP printer Threads: 1 Timeout: 1s	

From the Inventory tasks list, you'll always have the option to edit or delete a task. Some task details are shown in the configuration column. You can now select the task and ask a run. You can also enable or disable the task. The scheduling will only be used when the task is enabled.

Run an inventory task

An inventory task can be run in 2 cases:

1. manually after it has been selected in the **Inventory tasks** list and the **Run task** button is clicked
2. after the task has been enabled (select the task and click on **Enable**) and the agent finds the task scheduling configuration triggers the task

The screenshot shows the 'Inventory tasks' interface after the task has been run. The task 'Scan my printer' is now checked. A green bar at the bottom of the task row indicates 'Completed'. The 'Last run date' is now 'Fri Oct 20 19:19:25 2023'. The configuration and description remain the same.

Task name	Type	Scheduling	Last run date	Next run date	Configuration	Description
Scan my printer	Network scan	Every 24 hours	Fri Oct 20 19:19:25 2023	Disabled task	Target: server0 IP range: My SNMP printer Threads: 1 Timeout: 1s	

After a task has been run, you can see a progression bar. You can click on the eye or report icons to develop the task and access more details or features:

Inventory tasks

From 1 to 1 of 1 Display (number of items): 30

Task name	Type	Scheduling	Last run date	Next run date	Configuration	Description
☒ Scan my printer	Network scan	Every 24 hours	Fri Oct 20 19:19:25 2023	Disabled task	Target: server0 IP range: My SNMP printer Threads: 1 Timeout: 1s	

Completed

Created inventories: 0 Scanned IPs: 1 Devices with SNMP support: 0 IPs without SNMP response: 0 IPs without PING response: 0

Show < Scan my printer > task log Verbosity: debug

```
[info] Starting Scan my printer network scan task job...
[info] Running Scan my printer task...
[info] Can't load Net::NMBName, netbios can't be used
[debug] initializing job 1
[debug] initializing block 192.168.1.122-192.168.1.122
[debug] Current netdiscovery run expiration timeout: soon
[debug] using 1 netdiscovery worker
[debug] starting job 1 with 1 ip to scan using 1 worker
[debug] #1, scanning 192.168.1.122
[debug] #1, full match for sysobjectID .1.3.6.1.4.1.11.2.3.9.1 in database
[debug] #1, sysobjectID match: hp-peripheral mib support enabled
[debug] #1, - scanning 192.168.1.122 with SNMP, credentials [public-v1]: success
[debug] #1, - scanning 192.168.1.122 with echo ping: success
[debug] #1, - scanning 192.168.1.122 in arp table: success
[debug] #1, Current netinventory run expiration timeout: 1.0 hour
[debug] #1, using 1 netinventory worker
[debug] #1, full snmp scan of 192.168.1.122 with credentials [public-v1]
[debug] #1, full match for sysobjectID .1.3.6.1.4.1.11.2.3.9.1 in database
[debug] #1, sysobjectID match: hp-peripheral mib support enabled
[debug] #1, Netinventory worker terminated
[debug] #1, #1, worker termination
[debug] #1, Netdiscovery worker terminated
[info] #1, Scan my printer: Scan my printer network scan done
[debug] #1, Task run in 35.023 seconds
```

Update an inventory task

To update an inventory task, you simply can click on the Task name in the **Inventory task** list page.

You obtain the same form as for *inventory task creation*. And from here, you can:

1. Rename the inventory task
2. Change the target
3. Change the used scheduling
4. Change the tag
5. Change the ip range if the task type in **Network scan**
6. Click on Update to save your changes
7. Click on Go back to list or on Inventory tasks in the navigation bar to return to the list.

Delete an inventory task

For inventory task deletion, from the Inventory tasks list, you simply have to click on the related checkbox, and click on the Delete button.

Note

You can also prefer to just disable the task and only delete it after you're sure it won't be used anymore.

1.5.2.6.6 Configuration files

There are few files used to configure **ToolBox**:

- `toolbox-plugin.cfg`: This file permits to setup if and how the GLPI-Agent ToolBox plugin integration
- `toolbox.yaml`: This YAML file will contains a configuration section to tune your **ToolBox** experience but it will also contain your inventory, credentials, ip ranges & scheduling configurations. As such this file **MUST** be secured as much as possible as it will include very sensible data like you defined credentials.

Note

The `toolbox.yaml` file can be backed up when clicking on the Backup YAML button in the ToolBox plugin Configuration page. This eventually creates a backup folder at the same level if it doesn't exist. And it creates a copy renamed with a timestamp in that backup sub-folder.

A **container** can also be setup to support *Custom Fields* but you can also just copy the file downloaded from [GLPI Fields plugin](#) and select it in the dedicated *Custom fields YAML file* entry in configuration page. Be aware, this feature only make sens if you planned to edit locally stored *network inventories* before injecting it manually to a GLPI server. This feature has been developed to help people needing to scan a private network. It permits to edit manually few custom datas before uploading.

1.6 IDS Databases

The agent integrates IDS databases which are located in agent data directory. It consists of the following files:

- `pci.ids` is a database of PCI devices, used by local inventory module,
- `usb.ids` is a database of USB devices, used by local inventory module,
- `edid.ids` is a database of Screen manufacturer, used by local inventory module,
- `sysobject.ids` is a database of SNMP devices, used by network discovery and network inventory tasks modules.

Those files can easily be customized if needed, as their format is self-documented. However, local modifications will get lost on upgrade.

1.6.1 SNMP device IDS database

The `sysobject.ids` file is a database of known SNMP devices, indexed by the discriminant part of their `sysObjectID`¹ value:

```
9.1.1111    Cisco    NETWORKING    Catalyst 3500
+          +          +          +
|          |          |          |
|          |          |          +--> device model
|          |          +-----> device type
|          +-----> device manufacturer
+-----> sysObjectID model-specific suffix
```

The **sysObjectID model-specific suffix** is the last part of the full `sysObjectID` value, ie:

```
.1.3.6.1.4.1.9.1.111
+          +
|          |
|          +-----> model-specific suffix
+-----> shared prefix
```

¹ See `sysObjectID` definition in [RFC 3418](#):

`sysObjectID` is the vendor's authoritative identification of the network management subsystem contained in the entity. This value is allocated within the SMI enterprises subtree (1.3.6.1.4.1) and provides an easy and unambiguous means for determining *what kind of box* is being managed.

The sysObjectID value for any SNMP device can be retrieved by any SNMP client, using its OID (.1.3.6.1.2.1.1.2.0), or with either *Network inventory* or *Network discovery* command-line tools, with --debug flag.

Hint

You can contribute your modifications on the [related github sysobject.ids project](#).

1.7 Bug reporting

Attention

Problems related to this documentation should be reported on its [dedicated project issue tracker](#). You can also just edit the project in your own github environment and [submit a pull request](#).

For any GLPI agent problem or issue, you need to use [GitHub issue tracker](#), and report your issue with all the requested informations. This requires an GitHub user account.

Hint

Before reporting your problem, always take a look at the [opened and closed GitHub issues](#) by updating the *Filters* field and check if your problem has still not been addressed. You should also search for [any GLPI-Agent discussion](#) that could be related to your problem.

1.7.1 Problem description

You have to use english, for the sake of internationalization.

Always describe your problem clearly and precisely, for someone without any prior knowledge of your environment.

To do so, always describe:

- the operating system on which the GLPI agent is running,
- the GLPI agent version you're using and the way you installed it,
- the problem itself, eventually including log extract or agent run output,
- what you're expecting

Hint

You can obtain a more verbose output when running the agent from the console by using --debug option to set debug level 1, or by using --debug --debug options to set debug level 2. You can also set the *debug parameter* in configuration to 1 or 2 and restart the daemon or service.

1.7.2 External data

When the issue is related to some missing or invalid value, it can be caused by a parsing error while processing external data (a command output, a file, whatever...). In order to reproduce the problem and find a way to fix it, we need a sample of those data.

You can then attach the relevant file or command output as an attachment to the issue report. If there is any privacy concern, try to obfuscate sensible data or ask for an email where to send it.

On unix/linux, when running any command for such purpose, you always must unset locales first to avoid localized output:

```
$ export LC_ALL=C
```

Here are other command-specific advices:

1.7.2.1 WMI queries

Windows WMI queries can be exported with `wmic` tool:

```
C: > wmic path <somequery> get /Format:list > <somefile.txt>
```

1.7.2.2 Registry extract

Windows registry content can be exported with `regedit` tool:

```
C: > regedit /e <somefile.reg> <somekey>
```

1.7.2.3 dmidecode output

`dmidecode` output can be generated on any system, including windows, as we ship a `dmidecode` executable in the agent windows packaging, under the `perl\bin` subdirectory.

```
C: > "C: Files-Agent-agent" > dmidecode-output.txt
```

On unix/linux, just run the command in a root console:

```
$ dmidecode >dmidecode-output.txt
```

1.7.2.4 snmpwalk output

`Snmpwalk` output can be generated with the following command under unix/linux:

```
$ snmpwalk -v <version> -c <community> -t 15 -Cc -On -Ox <somehost> .1 > walk-filename.walk
```

Using an explicit root OID (.1 here), a non-default timeout (15 seconds, the same one as the agent default), and disabling internal consistency checks (-Cc) are required to extract all possible data.

Option `-Ox` is not mandatory but can help to enhance debugging discovery and inventory tasks as we may not know anything about the related MIB. So just having full numeric values can help.

Option `-On` is required to keep OID numerically to be sure to have a fully compliant snmp walk.

Hint

Joining known private MIBs related to your device could be really useful. You always can share them privately if you don't have the right to make them public.

Hint

When reporting a snmp walk, list all possible expected data you can know by another way, and at least the missing ones, like:

- serial number
- accurate model name

- manufacturer
- device name
- device mac address
- ...

1.8 Man pages

1.8.1 glpi-agent

Hint

Configuration options are also documented on the [Configuration](#) page.

Attention

--assetname-support option is available since glpi-agent 1.5
 --full-inventory-postpone and --full options are available since glpi-agent 1.8
 --oauth-client-id and --oauth-client-secret options are available since glpi-agent 1.10
 --glpi-version option is available since glpi-agent 1.12
 --required-category, --itemtype and --esx-itemtype ` options are available since glpi-agent 1.13
 --proxy option supports none value to not use environment variable since glpi-agent 1.13
 --ssl-cert-file and --ssl-key-file options support is available since glpi-agent 1.16

1.8.1.1 NAME

glpi-agent - GLPI perl agent For Linux/UNIX, Windows and MacOSX

1.8.1.2 SYNOPSIS

glpi-agent [options] [--server server|--local path]

Target definition options:

-s --server=URI	send tasks result to a server
-l --local=PATH	write tasks results locally

Target scheduling options:

--delaytime=LIMIT	maximum delay before first target, in seconds (3600). It also defines the maximum delay on network error. Delay on network error starts from 60, is doubled at each new failed attempt until reaching max
--lazy	do not contact the target before next scheduled time
--set-forcerun	set persistent state 'forcerun' option so a run will be started immediately during a start or init

(continues on next page)

(continued from previous page)

Task selection options:

```
--list-tasks           list available tasks and exit
--no-task=TASK[,TASK]... do not run given task
--tasks=TASK1[,TASK]...[,...] run given tasks in given order
```

Inventory task specific options:

```
--no-category=CATEGORY do not list given category items
--list-categories       list supported categories
--scan-homedirs         scan user home directories (false)
--scan-profiles         scan user profiles (false)
--html                  save the inventory as HTML (false)
--json                  save the inventory as JSON (false)
-f --force              always send data to server (false)
--backend-collect-timeout=TIME timeout for inventory modules execution (180)
--additional-content=FILE additional inventory content file
--assetname-support=1|2 [unix/linux only] set the asset name depending on the
↳ given value:
    - 1 (the default), the short hostname is used as asset
↳ name
    - 2, the as-is hostname (can be fqdn) is used as asset
↳ name
this feature is not supported on MacOS or Windows
--partial=CATEGORY     make a partial inventory of given category
                        items, this option implies --json
--credentials           set credentials to support database inventory
--full-inventory-postpone=NUM set number of possible full inventory postpone (14)
--full                  force inventory task to generate a full inventory
--required-category=CATEGORY list of category required even when postponing full
↳ inventory
--itemtype=TYPE         set asset type for target supporting genericity like
↳ GLPI 11+
Remark: This option is also used by RemoteInventory task
```

ESX task specific options:

```
--esx-itemtype=TYPE    set ESX asset type for target supporting genericity
↳ like GLPI 11+
```

RemoteInventory task specific options:

```
--remote=REMOTE[,REMOTE]... specify a list of remotes to process in place
of remotes managed via glpi-remote command
--remote-workers=COUNT maximum number of workers for remoteinventory task
```

Package deployment task specific options:

```
--no-p2p               do not use peer to peer to download
                        files (false)
```

Network options:

```
-P --proxy=PROXY       proxy address
-u --user=USER          user name for server authentication
-p --password=PASSWORD password for server authentication
--ca-cert-dir=DIRECTORY CA certificates directory
```

(continues on next page)

(continued from previous page)

```

--ca-cert-file=FILE          CA certificates file
--no-ssl-check                do not check server SSL certificate
                              (false)
--ssl-fingerprint=FINGERPRINT Trust server certificate if its SSL fingerprint
                              matches the given one
-C --no-compression           do not compress communication with server
                              (false)
--timeout=TIME                connection timeout, in seconds (180)

--ssl-cert-file=FILE          ssl client certificate file
--ssl-key-file=FILE           ssl client private key file
                              (assumed included in cert file if missing)

Web interface options:
--no-httpd                    disable embedded web server (false)
--httpd-ip=IP                 network interface to listen to (all)
--httpd-port=PORT             network port to listen to (62354)
--httpd-trust=IP              trust requests without authentication
                              token (false)
--listen                       enable listener target if no local or
                              server target is defined

Server authentication:
--oauth-client-id=ID           oauth client id to request oauth access token
--oauth-client-secret=SECRET   oauth client secret to request oauth access token

Logging options:
--logger=BACKEND              logger backend (stderr)
--logfile=FILE                 log file
--logfile-maxsize=SIZE         maximum size of the log file in MB (0)
--logfacility=FACILITY          syslog facility (LOG_USER)
--color                        use color in the console (false)

Configuration options:
--config=BACKEND               configuration backend
--conf-file=FILE               configuration file
--conf-reload-interval=<SECONDS> number of seconds between two
                              configuration reloadings

Execution mode options:
-w --wait=LIMIT                maximum delay before execution,
                              in seconds
-d --daemon                    run the agent as a daemon (false)
--no-fork                       don't fork in background (false)
-t --tag=TAG                    add given tag to inventory results
--debug                         debug mode (false)
--setup                         print the agent setup directories
                              and exit
--vardir=PATH                  use specified path as storage folder for agent
                              persistent datas

--glpi-version=<VERSION>       set targeted glpi version to enable supported features

```

(continues on next page)

(continued from previous page)

```
--version           print the version and exit
--no-win32-ole-workaround [win32 only] disable win32 work-around
                        used to better handle Win32::OLE apis.
                        !!! Use it at your own risk as you may
                        experiment perl crash under win32 !!!
```

1.8.1.3 DESCRIPTION

The *glpi-agent* agent is a generic multi-platform agent. It can perform a large array of management tasks, such as local inventory, software deployment or network discovery. It can be used either standalone, or in combination with a compatible server acting as a centralized control point.

1.8.1.4 OPTIONS

Most of the options are available in a *short* form and a *long* form. For example, the two lines below are all equivalent:

```
% glpi-agent -s localhost
% glpi-agent --server localhost
```

1.8.1.4.1 Target definition options

-s, --server=URI

Send the results of tasks execution to given server.

If *URI* doesn't start with <http://> or <https://>, the agent assume the parameter is a hostname and rewrite it as:

```
% --server=http://my-glpi-server/
```

In general, GLPI server URL have this format:

```
http://my-glpi-server/
```

and FusionInventory for GLPI this one:

```
http://my-glpi-server/plugins/fusioninventory
```

Multiple values can be specified, using comma as a separator.

-l, --local=PATH

Write the results of tasks execution locally.

Exact behaviour according to given path:

- if *PATH* is a directory, a file will be created therein
- if *PATH* is a file, it will be used directly
- if *PATH* is '-', STDOUT will be used

Multiple values can be specified, using comma as a separator.

1.8.1.4.2 Target scheduling options

--delaytime=LIMIT

Set an initial delay before the first target, whose value is computed randomly between $LIMIT / 2$ and *LIMIT* seconds. This setting is ignored for server targets after the initial contact, in favor of server-specified parameter (*PROLOG_FREQ*).

--lazy

Do not contact the target before next scheduled time.

This option is only available when the agent is not run as a server.

1.8.1.4.3 Task selection options**--list-tasks**

List all available tasks, tasks planned for execution and exit

--no-task=TASK

Do not run given task.

Multiple values can be specified, using comma as a separator. See option *--list-tasks* for the list of available tasks.

--tasks=TASK

Run given tasks in given order.

Multiple tasks can be specified, using comma as a separator. A task can be specified several times. if '...' is given as last element, all other available tasks are executed.

See option *--list-tasks* for the list of available tasks.

Examples :

- **--tasks=inventory,deploy,inventory**

First task executed is 'inventory', second task is 'deploy', third and last task is 'inventory'.

- **--tasks=inventory,deploy,...**

First executed task is 'inventory', second task is 'deploy' and then all other available tasks are executed.

1.8.1.4.4 Inventory task specific options**--no-category=CATEGORY**

Do not list given category items in inventory.

Multiple values can be specified, using comma as a separator. The available categories are:

- accesslog
- antivirus
- battery
- bios
- controller
- cpu
- database
- drive
- environment
- firewall
- hardware
- input
- licenseinfo
- local_group

- local_user
- lvm
- memory
- modem
- monitor
- network
- os
- port
- printer
- process
- provider
- psu
- registry
- remote_mgmt
- rudder
- slot
- software
- sound
- storage
- usb
- user
- video
- virtualmachine

--list-categories

List all supported categories by scanning all available inventory modules

--credentials=*CREDENTIALS*

Setup credentials for database inventory

CREDENTIALS should be a list of "key:value" separated by commas like in: For example: `--credentials="type:login_password,login:root,password:*****,use:postgresql,params_id:0"`

--full-inventory-postpone=*NUM*

Set the number of time the agent can decide to generate a partial inventory with only changed category before generating a full inventory.

--full

Force inventory task to generate a full inventory even if **full-inventory-postpone** option is set. Indeed this is equivalent to set **--full-inventory-postpone=0**.

--required-category=*CATEGORY*

Force inventory task to always include given category if **full-inventory-postpone** option is set and the current inventory task run involves to generate a partial inventory.

Multiple values can be specified, using comma as a separator. List of categories is the same than the one for `--no-category` option, but *bios* and *hardware* categories are always implied and can be omitted as they are still required for normal inventory import.

--itemtype=TYPE

Allow to set JSON inventory itemtype to *TYPE*. This feature requires a target supporting genericity, like GLPI 11+.

When expected asset itemtype in GLPI 11+ is **Server**, *itemtype* option value must be set to **Glpi\CustomAsset\ServerAsset**.

Note: The suffix **Asset** must always be added at the end of the class name to avoid conflicts with PHP reserved keywords.

--scan-homedirs

Allow the agent to scan home directories for virtual machines.

--scan-profiles

Allow the agent to scan user profiles for software.

--html|--json

Save the inventory as HTML or JSON.

This is only used for local inventories.

-f, --force

Send an inventory to the server, even if this last one doesn't ask for it.

--backend-collect-timeout=TIME

Timeout for inventory modules execution.

--additional-content=FILE

Additional inventory content file.

This file should be an XML file, using same syntax as the one produced by the agent.

1.8.1.4.5 ESX task specific options

--esx-itemtype=TYPE

Allow to set ESX JSON inventory itemtype to *TYPE*. This feature requires a target supporting genericity, like GLPI 11+.

When expected ESX asset itemtype in GLPI 11+ is **Esx**, *esx-itemtype* option value must be set to **Glpi\CustomAsset\EsxAsset**.

1.8.1.4.6 Package deployment task specific options

--no-p2p

Do not use peer to peer to download files.

1.8.1.4.7 Server target specific options

-P, --proxy=PROXY

Use *PROXY* as HTTP proxy.

By default, the agent uses HTTP_PROXY environment variable unless option value is defined to **none**.

-u USER, --user=USER

Use *USER* for server authentication.

-p, --password=PASSWORD

Use *PASSWORD* for server authentication.

--ca-cert-dir=*DIRECTORY*

CA certificates directory.

--ca-cert-file=*FILE*

CA certificates file.

--no-ssl-check

Do not check server SSL certificate.

--timeout=*TIME*

Timeout for server connections.

--ssl-cert-file=*FILE*

SSL client certificate filename.

--ssl-key-file=*FILE*

SSL client private key filename. If missing, assumed is included in cert file

1.8.1.4.8 Web interface options

--no-httpd

Disable the embedded web server.

--httpd-ip=*IP*

The network interface to use for the embedded web server (all).

--httpd-port=*PORT*

The network port to use for the embedded web server (62354).

--httpd-trust=*IP*

Trust requests from given addresses without authentication token (false).

For example: "192.168.0.0/24", "192.168.168.0.5" or an IP range like "20.34.101.207 - 201.3.9.99". Hostnames are also accepted. See [Net::IP](#) documentation to get more example.

Multiple values can be specified, using comma as a separator.

--listen

This option should be used if no local or server target is defined and the agent still needs to answer http requests.

--no-httpd should not be set and **--httpd-trust** should be set to enable trusted remote clients.

1.8.1.4.9 Server authentication

--oauth-client-id=*ID*

The OAuth client id required to authenticate against GLPI >= 11.

--oauth-client-secret=*SECRET*

The OAuth client secret required to authenticate against GLPI >= 11.

1.8.1.4.10 Logging options

--logger=*BACKEND*

Logger backend to use.

Multiple values can be specified, using comma as a separator. The available backends are:

- stderr: log messages directly in the console.
- file: log messages in a file.
- syslog: log messages through the local syslog server.

Multiple values can be specified, using comma as a separator.

--logfile=FILE

Log message in *FILE* (implies File logger backend).

--logfile-maxsize=SIZE

Max logfile size in MB, default is unlimited. When the max size is reached, the file is truncated. This is only useful if there is no log rotation mechanism on the system.

--logfacility=FACILITY

Syslog facility to use (default LOG_USER).

--color

Display color on the terminal, when the Stderr backend is used.

This options is ignored on Windows.

1.8.1.4.11 Configuration options

--config=BACKEND

Configuration backend to use.

The available backends are:

- file: read configuration from a file (default anywhere else as Windows).
- registry: read configuration from the registry (default on Windows).
- none: don't read any configuration.

--conf-file=FILE

Use *FILE* as configuration file (implies file configuration backend).

--conf-reload-interval=SECONDS

SECONDS is the number of seconds between two configuration reloadings. Default value is 0, which means that configuration is never reloaded. Minimum value is 60. If given value is less than this minimum, it is set to this minimum. If given value is less than 0, it is set to 0.

1.8.1.4.12 Execution mode options

-w LIMIT, --wait=LIMIT

Wait a random delay whose value is computed randomly between 0 and LIMIT seconds, before execution. This is useful when execution is triggered from some kind of system scheduling on multiple clients, to spread the server load.

-d, --daemon

Run the agent as a daemon.

--no-fork

Don't fork in background.

This is only useful when running as a daemon.

--pidfile[=FILE]

Store pid in *FILE* or in default PID file.

This is only useful when running as a daemon and still not managed with a system service manager like systemd.

--tag=TAG

Add the given tag to every inventory results.

--debug

Turn the debug mode on. You can use the parameter up to 3 times in a row to increase the verbosity (e.g: **--debug --debug --debug**).

Level 3 turns on the debug mode of some external libraries like [Net::SSLeay](#). These messages will only be printed on STDERR.

--setup

Print the agent setup directories and exit.

--version

Print the version and exit.

1.8.2 glpi-inventory

Attention

--assetname-support options is available since glpi-agent 1.5

--glpi-version option is available since glpi-agent 1.12

1.8.2.1 NAME

glpi-inventory - Standalone inventory

1.8.2.2 SYNOPSIS

glpi-inventory [options]

Options:

--scan-homedirs	scan use home directories (false)
--scan-profiles	scan user profiles (false)
--html	save the inventory as HTML (false)
--json	save the inventory as JSON (false)
--no-category=CATEGORY	do not list given category items
--partial=CATEGORY	make a partial inventory of given category items, this option implies --json
--required-category=CATEGORY	list of category required even when generating a
--partial inventory	
--credentials	set credentials to support database inventory
-t --tag=TAG	mark the machine with given tag
--backend-collect-timeout=TIME	timeout for inventory modules execution (30)
--glpi-version=VERSION	set targeted glpi version to enable supported features
--itemtype=TYPE	set asset type for target supporting genericity like
--GLPI 11+	
--additional-content=FILE	additional inventory content file
--assetname-support=1 2	[unix/linux only] set the asset name depending on the
--given value:	
--name	- 1 (the default), the short hostname is used as asset
--name	- 2, the as-is hostname (can be fqdn) is used as asset
--name	
--verbose	this feature is not supported on MacOS or Windows verbose output (control messages)

(continues on next page)

(continued from previous page)

<code>--debug</code>	debug output (execution traces)
<code>-h --help</code>	print this message and exit
<code>--version</code>	print the task version and exit

1.8.2.3 DESCRIPTION

glpi-inventory can be used to run an inventory task without a GLPI server.

1.8.3 glpi-netdiscovery

Attention

`--v1` and `--v2c` options are available since *glpi-agent* 1.5
`--backend-collect-timeout` option is available since *glpi-agent* 1.6
`--retries` option is available since *glpi-agent* 1.9
`--glpi-version` option is available since *glpi-agent* 1.16

1.8.3.1 NAME

glpi-netdiscovery - Standalone network discovery

1.8.3.2 SYNOPSIS

glpi-netdiscovery [options] --first <address> --last <address>

Options:

<code>--host <ADDRESS></code>	Host IP address to scan or IP range first address
<code>--first <ADDRESS></code>	IP range first address
<code>--last <ADDRESS></code>	IP range last address
<code>--port <PORT[,PORT2]></code>	SNMP port (161)
<code>--protocol <PROT[,P2]></code>	SNMP protocol/domain (udp/ipv4)
<code>--community <STRING></code>	SNMP community string (public)
<code>--v1</code>	select SNMP version 1 (the default)
<code>--v2c</code>	select SNMP version 2c (1 by default)
<code>--credentials <STRING></code>	SNMP credentials (version:1,community:public)
<code>--timeout <TIME></code>	SNMP timeout, in seconds (1)
<code>--retries</code>	SNMP requests maximum retries (0)
<code>--backend-collect-timeout <TIME></code>	base expiration timeout, in seconds (180)
<code>--entity <ENTITY></code>	GLPI entity
<code>--threads <COUNT></code>	number of discovery threads (1)
<code>--glpi-version VERSION</code>	set targeted glpi version to enable supported features
<code>--control</code>	output control messages
<code>--file <FILE></code>	snmpwalk input file
<code>-i --inventory</code>	chain with netinventory task for discovered devices
<code>-s --save <FOLDER></code>	base folder where to save discovery and inventory xmls - netdiscovery xmls will go in <FOLDER>/netdiscovery - netinventory xmls will go in <FOLDER>/netinventory
<code>--debug</code>	debug output

(continues on next page)

(continued from previous page)

<code>-h --help</code>	print this message and exit
<code>--version</code>	print the task version and exit

1.8.3.3 DESCRIPTION

glpi-netdiscovery can be used to run a network discovery task without a GLPI server.

1.8.3.4 OPTIONS

--first|--host *ADDRESS*

Set the first IP address of the network range to scan.

--last *ADDRESS*

Set the last IP address of the network range to scan.

If not set, it is set with the value of the `--first` or `--host` option.

--port *PORT[,PORT2]*

List of ports to try, defaults to: 161

Set it to 161,16100 to first try on default port and then on 16100.

--protocol *PROTOCOL[,PROTOCOL2]*

List of protocols to try, defaults to: udp/ipv4

Possible values are: udp/ipv4,udp/ipv6,tcp/ipv4,tcp/ipv6

--file *FILE*

Run an offline discovery against snmpwalk output, stored in the given file.

If no host or first ip is provided, ip is set to emulate 1.1.1.1 ip scan.

--community *STRING*

Use given string as SNMP community (assume SNMPv1). This option can be used multiple times to try different communities.

--v1

Use SNMP v1. This is the default, but you can use the option to try SNMP v1 & SNMP v2c.

--v2c

Use SNMP v2c. Can be used in combination with `--v1` to try the 2 versions.

--credentials *STRING*

Use given string as SNMP credentials specification. This specification is a comma-separated list of key:value authentication parameters, such as:

- version:2c,community:public
- version:3,username:admin,authpassword:s3cr3t,privpassword:s3cr3t
- etc.

Supported keys are:

- version with value set to 1, 2c or 3

In the case version is set to 1 or 2c:

- community

In the case version is set to 3:

- username (required)

- authpassword
- authprotocol with value set to md5 (the default if not set) or sha
- privpassword (required if authpassword is set)
- privprotocol with value set to des (the default if not set), aes or 3des

--timeout *TIME*

Set SNMP timeout, in seconds.

--retries *NUMBER*

Set maximum number of retries a SNMP request can be sent again after no response.

--backend-collect-timeout *TIME*

Set base expiration timeout, in seconds. Global task expiration will depend on the number of ips.

--entity *ENTITY*

Set GLPI entity.

--threads *COUNT*

Use given number of inventory threads.

--control

Output server-agent control messages, in addition to inventory result itself.

--debug

Turn the debug mode on. Multiple usage allowed, for additional verbosity.

1.8.3.5 EXAMPLES

Run a discovery against a network range, using SNMP version 1:

```
$> glpi-netdiscovery --first 192.168.0.1 --last 192.168.0.254 --community public
```

Run a discovery against a network range, using multiple SNMP credentials:

```
$> glpi-netdiscovery --first 192.168.0.1 --last 192.168.0.254 \  
--credentials version:2c,community:public \  
--credentials version:3,username:admin,authpassword:s3cr3t,privpassword:s3cr3t
```

Emulate discovery using a snmpwalk file:

```
$> glpi-netdiscovery --file device.walk
```

1.8.4 glpi-netinventory

⚠ Attention

- v1 and --v2c options are available since glpi-agent 1.5
- backend-collect-timeout option is available since glpi-agent 1.6
- retries option is available since glpi-agent 1.9
- glpi_version option is available since glpi-agent 1.12
- glpi-version option replaces --glpi_version since glpi-agent 1.16

1.8.4.1 NAME

glpi-netinventory - Standalone network inventory

1.8.4.2 SYNOPSIS

glpi-netinventory [options] [--host <host>|--file <file>]

Options:

```
--host <HOST>           target host
--port <PORT[,PORT2]>    SNMP port (161)
--protocol <PROT[,P2]>   SNMP protocol/domain (udp/ipv4)
--file <FILE>           snmpwalk output file
--community <STRING>    community string (public)
--credentials <STRING>   SNMP credentials (version:1,community:public)
--timeout <TIME>         SNMP timeout, in seconds (15)
--retries                SNMP requests maximum retries (0)
--backend-collect-timeout <TIME>
                        base expiration timeout, in seconds (180)
--glpi-version VERSION   set targeted glpi version to enable supported features
--type <TYPE>           force device type
--threads <COUNT>      number of inventory threads (1)
--control                output control messages
--debug                 debug output
-h --help               print this message and exit
--version               print the task version and exit
```

1.8.4.3 DESCRIPTION

glpi-netinventory can be used to run a network inventory task without a GLPI server.

1.8.4.4 OPTIONS

--host *HOST*

Run an online inventory against given host. Multiple usage allowed, for multiple hosts.

--port *PORT[,PORT2]*

List of ports to try, defaults to: 161

Set it to 161,16100 to first try on default port and then on 16100.

--protocol *PROTOCOL[,PROTOCOL2]*

List of protocols to try, defaults to: udp/ipv4

Possible values are: udp/ipv4,udp/ipv6,tcp/ipv4,tcp/ipv6

--file *FILE*

Run an offline inventory against snmpwalk output, stored in given file. Multiple usage allowed, for multiple files.

--community *STRING*

Use given string as SNMP community (assume SNMPv1)

--v1

Use SNMP v1. This is the default.

--v2c

Use SNMP v2c.

--credentials *STRING*

Use given string as SNMP credentials specification. This specification is a comma-separated list of key:value authentication parameters, such as:

- version:2c,community:public
- version:3,username:admin,authpassword:s3cr3t,privpassword:s3cr3t
- etc.

Supported keys are:

- version with value set to 1, 2c or 3

In the case version is set to 1 or 2c:

- community

In the case version is set to 3:

- username (required)
- authpassword
- authprotocol with value set to md5 (the default if not set) or sha
- privpassword (required if authpassword is set)
- privprotocol with value set to des (the default if not set), aes or 3des

--timeout *TIME*

Set SNMP timeout, in seconds.

--retries *NUMBER*

Set maximum number of retries a SNMP request can be sent again after no response.

--backend-collect-timeout *TIME*

Set base expiration timeout, in seconds. It is used to set one device scan: 180 by default, means 900 (5x180) by device.

--type *TYPE*

Force device type, instead of relying on automatic identification. Currently allowed types:

- COMPUTER
- NETWORKING
- PRINTER
- STORAGE
- POWER
- PHONE

--threads *count*

Use given number of inventory threads.

--control

Output server-agent control messages, in addition to inventory result itself.

--debug

Turn the debug mode on. Multiple usage allowed, for additional verbosity.

1.8.4.5 EXAMPLES

Run an inventory against a network device, using SNMP version 2c authentication:

```
$> glpi-netinventory --host 192.168.0.1 --credentials version:2c,community:public
```

Run an inventory against a network device, using SNMP version 3 authentication and forcing its type:

```
$> glpi-netinventory --host my.device --type NETWORKING \
--credentials version:3,username:admin,authpassword:s3cr3t,privpassword:s3cr3t
```

1.8.5 glpi-esx

Attention

Some informations on this man page are probably outdated.

Since glpi-agent 1.5, `--directory` option is deprecated, you should use `--path` new option instead.

`--timeout` option is available since glpi-agent 1.7

`--glpi-version` option is available since glpi-agent 1.12

1.8.5.1 NAME

glpi-esx - vCenter/ESX/ESXi remote inventory from command line

1.8.5.2 SYNOPSIS

glpi-esx --host <host> --user <user> --password <password> --path <directory or file>

Options:

<code>--help</code>	this menu
<code>--debug</code>	debug mode (false)
<code>--host hostname</code>	ESX server hostname
<code>--user username</code>	user name
<code>--password xxxx</code>	user password
<code>--path path</code>	output directory or file
<code>-t --timeout sec</code>	connection attempt timeout in seconds (defaults to 180)
<code>--stdout</code>	dump inventory on stdout (enabled by default if no path is set)
<code>--tag tag</code>	tag for the inventoried machine
<code>--json</code>	use json as format for generated inventories
<code>--glpi-version VERSION</code>	set targeted glpi version to enable supported features
<code>--esx-itemtype=TYPE</code>	set ESX asset type for target supporting genericity like GLPI

↪ 11+

Advanced options:

<code>--dump</code>	also dump esx host full info datas in a <code>*-hostfullinfo.dump</code> file
<code>--dumpfile file</code>	generate one inventory from a <code>*-hostfullinfo.dump</code> file

1.8.5.3 EXAMPLES

```
% glpi-esx --host myesx --user foo --password bar --path /tmp --json
```

You can import the .json file in your inventory server with the glpi-injector tool.

```
% glpi-injector -v --file /tmp/*.json -u https://example/
```

1.8.5.4 DESCRIPTION

glpi-esx creates inventory of remote ESX/ESXi and vCenter VMware. It uses the SOAP interface of the remote server.

Supported systems start from:

ESX and ESXi 3.5

ESX and ESXi 4.1

ESXi 5.0

vCenter 4.1

vCenter 5.0

Active Directory users, please note the AD authentication doesn't work. You must create a account on the VMware server.

1.8.5.5 LIMITATION

So far, ESX serial number are not collected.

1.8.5.6 SECURITY

The SSL hostname check of the server is disabled.

1.8.6 glpi-injector

Attention

--proxy options is available since glpi-agent 1.5

--oauth-client-id and --oauth-client-secret options are available since glpi-agent 1.10

--ca-cert-file, --ssl-fingerprint and --ssl-key-file options are available since glpi-agent 1.16

--debug --no-ssl-check options with glpi-injector can be used since glpi-agent 1.16 to show server --ssl-fingerprint possible value.

1.8.6.1 NAME

glpi-injector - A tool to push inventory in an OCS Inventory or compatible server.

1.8.6.2 SYNOPSIS

```
glpi-injector [-h|--help] [-R|--recursive] [-r|--remove] [-v|--verbose] [--debug] [--useragent <user-agent>|-x|--xml-ua|-json-ua] [-C|--no-compression] [--no-ssl-check] [--ssl-cert-file <private certificate file>] [[-P|--proxy] <proxy url>] [--ca-cert-file <CA certificate file>] [--ss <CA certificate file>] [[-f|--file] <file>|[-d|--directory] <directory>|--stdin] [-u|--url] <url>
```

Options:

```

-h --help          this menu
-d --directory     load every inventory files from a directory
-R --recursive     recursively load inventory files from <directory>
-f --file          load a specific file
-u --url           server URL
-r --remove        remove succesfully injected files
-v --verbose       verbose mode
--debug           debug mode to output server answer
--stdin           read data from STDIN
--useragent       set used HTTP User-Agent for POST
-x --xml-ua --json-ua
                  use Client version found in XML or JSON as User-Agent for POST
--no-ssl-check     do not check server SSL certificate
--ca-cert-file=FILE CA certificates file
--ssl-cert-file=FILE client certificate file
--ssl-key-file=FILE client private key file (asumed included in cert file if missing)
--ssl-fingerprint=FINGERPRINT Trust server certificate on its SSL fingerprint
-C --no-compression don't compress sent XML inventories
-P --proxy=PROXY proxy address
--oauth-client-id
                  oauth client id to request oauth access token
--oauth-client-secret
                  oauth client secret to request oauth access token

```

Examples:

```

glpi-injector -v -f /tmp/toto-2010-09-10-11-42-22.json --url https://login:pw@example/
glpi-injector -v -R -d /srv/ftp/fusion --url https://login:pw@example/

```

1.8.6.3 DESCRIPTION

This tool can be used to test your server, do benchmark or push inventory from off-line machine.

1.8.7 glpi-remote

 Hint

The 'agent' sub command can only be used toward GLPI agent with *Inventory Server Plugin* enabled.

⚠ Attention

--stricthostkeychecking option is available since glpi-agent 1.7

--ssl-key-file option is available since glpi-agent 1.16

1.8.7.1 NAME

glpi-remote - A tool to scan, manage and initialize virtual remote agents

1.8.7.2 SYNOPSIS

glpi-remote [options] [--server server|--local path] [command] [command options]

Options:

```
-h --help                this menu
-t --timeout <SECS>     requests timeout in seconds (defaults to 10)
-p --port <LIST>         remote ports list to scan (defaults to '22,5985,5986')
--ssh                    connect using SSH
--ssl                    connect using SSL (winrm or with agent sub-command)
--no-ssl-check           do not check agent SSL certificate (winrm or agent sub-command)
--stricthostkeychecking <yes|no|off|accept-new|ask> (defaults to 'accept-new')
                        use given option when checking hostkey during ssh remote add
--ca-cert-dir <PATH>     CA certificates directory
--ca-cert-file <FILE>    CA certificates file (winrm or for agent sub-command)
--ssl-fingerprint <FINGERPRINT>
                        Trust server certificate if its SSL fingerprint matches the given
one
--ssl-cert-file <FILE>   Client certificate file (winrm)
--ssl-key-file <FILE>    Client private key file (assumed included in cert file if
missing)
-u --user                authentication user
-P --password            authentication password
-X --show-passwords     (list command) show password as they are masked by default
-c --credentials        credentials list for scan
-v --verbose             verbose mode
--debug                 debug mode
-C --no-check            don't check given remote is alive when adding
-i --inventory           don't register remotes, but run inventory on found remotes
-T --threads <NUM>      number of threads while scanning (defaults to 1)
-A --add                add scanned remotes to target so they always be inventoried
                        by RemoteInventory task
-U --useragent           set used HTTP User-Agent for requests
--vardir <PATH>         use specified path as storage folder for agent persistent datas
```

Target definition options:

```
-s --server=<URI>        agent will send tasks result to that server
-l --local=<PATH>        agent will write tasks results locally
--target=<TARGETID>     use target identified by its id (see list targets command)
```

Remote GLPI agent having inventory server plugin enabled options:

```
-b --baseurl <PATH>     remote base url if not /inventory
-K --token <TOKEN>      token as shared secret
-I --id <ID>            request id to identify requests in agent log
```

(continues on next page)

(continued from previous page)

```
--no-compression    ask to not compress sent XML inventories
```

Sub-commands

```
list [targets]      list known remotes or targets
add <url>+          add remote with given URL list
del[ete] <index|deviceid>+
                    delete remote with given list index or given deviceid or
                    current known one when alone or all remotes while using
                    __ALL__ as id
scan <first> [last] [TODO] scan given ip range for remote access or just <first> and
                    register it/them as remote agent
agent [hosts]       remotely claim an inventory to given remote hosts with a
                    GLPI agent having inventory server plugin enabled
                    (see https://glpi-agent.rtfid.io/inventory-server-plugin.html)
```

Supported environment variables:

```
USERNAME
PASSWORD
PORT
CA_CERT_PATH
CA_CERT_FILE
SSL_CERT_FILE
CREDENTIALS
```

Examples:

```
glpi-remote list
glpi-remote list targets
glpi-remote add ssh://admin:pass@192.168.43.237
glpi-remote add ssh://admin:pass@192.168.43.237 --stricthostkeychecking=no
glpi-remote add ssh://admin:pass@192.168.43.238 --no-check
glpi-remote add winrm://admin:pass@192.168.48.250 --no-check --target server0
glpi-remote delete 1
glpi-remote scan 192.168.43.1 192.168.43.254
glpi-remote scan 10.0.0.1 10.0.10.254 --inventory -s https://myglpi/
glpi-remote scan 10.0.0.1 10.0.10.254 --inventory -l /var/tmp/remotes
glpi-remote scan --inventory
glpi-remote scan 192.168.48.99 | glpi-injector --url https://myglpi/
```

Examples for agent command:

```
glpi-remote -K strong-shared-secret agent 192.168.43.236
glpi-remote -v -K strong-shared-secret agent 192.168.43.237 | \
    glpi-injector -url https://myglpi/
glpi-remote -K strong-shared-secret -d /var/remote agent 192.168.43.236 192.168.43.237
```

1.8.7.3 DESCRIPTION

The *glpi-remote* tool is used to manage virtual agents known locally by *glpi-agent*. A virtual agent is used to make remote inventories and is essentially defined by a remote access. A remote access can be defined by ssh authorization for unix/linux platforms or WinRM authorizations for a WinRM enabled platform like win32.

1.8.7.4 OPTIONS

Most of the options are available in a *short* form and a *long* form. For example, the two lines below are all equivalent:

```
% glpi-agent -s localhost
% glpi-agent --server localhost
```

1.8.7.4.1 Target definition options

-s, --server=*URI*

Send the results of tasks execution to given server.

Multiple values can be specified, using comma as a separator.

-l, --local=*PATH*

Write the results of tasks execution locally.

--target=*TARGETID*

Use the given TARGETID to look for the expected target for result submission.

For example, **server0** is the first server target setup in agent configuration.

Remark:

- target option is generally mandatory while adding remote or scanning for remotes
- if one server and only one is still setup in the agent it will be selected as default target
- when scanning and making inventory, uses any target option or each inventory will be sent to standard output

1.8.7.4.2 General options

-t, --timeout=*SECS*

Set the timeout for network requests (defaults to 10 seconds).

-p, --port=*LIST*

A list of ports used when making a scan and to discover remote computers. The defaults is to scan the standard ssh port and winrm ports: 22,5985,5986.

--ssh

Use ssh protocol for connection.

--ssl

Use SSL protocol for connecting with WinRM protocol or to a remote agent with inventory server plugin enabled.

--ca-cert-dir=*DIRECTORY*

CA certificates directory.

--ca-cert-file=*FILE*

CA certificates file.

--ssl-cert-file=*FILE*

SSL certificate file for authentication

--ssl-key-file=*FILE*

SSL client private key filename. If missing, assumed is included in cert file

--no-ssl-check

Do not check server SSL certificate.

-u *USER*, --user=*USER*

Use *USER* for remote authentication.

-P, --password=*PASSWORD*

Use *PASSWORD* for remote authentication.

-X, --show-passwords

By default, **list** sub-command won't show remotes passwords. This option asks to unmask them during remotes listing.

-c, --credentials=*LIST*

List of credentials to try during a scan.

-v, --verbose

Enable verbose mode.

--debug

Turn the debug mode on. You can use the parameter up to 2 times in a row to increase the verbosity (e.g: **--debug --debug**).

-C, --no-check

Don't check remote is alive while adding it.

-i, --inventory

Don't register remotes as they are discovered but just run the inventory task on them.

-T, --threads=*NUM*

Setup number of threads while doing a scan. By default, the agent only uses one thread.

-A, --add

Add discovered remotes to local remotes list.

-U, --useragent=*USER-AGENT*

Define HTTP user agent for request (mostly if required for winrm).

1.8.7.4.3 *agent* sub-command options

-b, --baseurl=*PATH*

Remote base url if the default */inventory* has been changed in the remote plugin configuration.

-K, --token=*TOKEN*

Shared secret required to request an inventory to the remote plugin.

-I, <--id>=*ID*

Request-ID to identify the request in the agent log.

--no-compression

Ask to skip requested inventory compression.

1.8.7.4.4 Sub-commands

- **list [targets]**

list known remotes or list targets

- **add url+**

add remote with given URL list

- **del[ete] index|deviceid+**

Delete remote with given:

- list index

- given deviceid

- **scan** *first* [*last*]

Scan given ip range for remote access or just *first* and register it/them as remote agent

- Remotely claim an inventory to given remote hosts with a GLPI agent having inventory server plugin enabled.

See online documentation for details: <https://glpi-agent.rtfid.io/inventory-server-plugin.html>

For security reasons, you can set few environment variables to store sensible datas.

- ### 1.8.8 glpi-win32-service

This script is only available on **win32**. It is the base script used to setup the agent as a service during **win32** installation. You don't normally need to run it by itself.

This script is also handy to setup manually a **win32** service based on a development environment.

glpi-win32-service - GLPI perl Agent service for Windows

```
gipi-win32-service [--register|--delete|--help] [options]
```

Register options:

(continues on next page)

(continued from previous page)

```
-d --displayname="Nice Name"    display name of the service
-l --libdir=PATH                full path to agent perl libraries use it if
                                not found by the script
-p --program="path to program" perl script to start as service
```

Delete options:

```
-n --name=NAME                  unique system name of the service to delete
```

Samples to use from sources base:

```
perl bin/glpi-win32-service --help
perl bin/glpi-win32-service --register
perl bin/glpi-win32-service --delete
perl bin/glpi-win32-service --register -n glpi-agent-test -d "[TEST] GLPI Agent Service
↪ "
perl bin/glpi-win32-service --delete -n glpi-agent-test
```

DOCUMENTATION LICENSE

This documentation is distributed under the terms of the [Creative Commons License Attribution-ShareAlike 4.0 \(CC BY-SA 4.0\)](#).

For the complete terms of the license, please refer to <https://creativecommons.org/licenses/by-sa/4.0/legalcode>.

You are free to:

- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

- Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.